TOOL GRAPH COMPLEXITY FOR LLM TOOL-CALLING: A REPRODUCIBLE BENCHMARK AUDITING AND GENERATION FRAMEWORK

Anonymous authors
Paper under double-blind review

ABSTRACT

Language-model agents are increasingly evaluated by how many tools a task requires: benchmarks stratify difficulty by tool count, and leaderboards report accuracy falling as tool lists grow. We argue that this number conflates two different things. One is a property of the benchmark: the reference tool set an agent is scored against often lists tools the task does not need, so recall penalizes any agent that finds a shorter correct plan. The other is a property of the task: how the required tools must compose. We introduce AGENTGRAPHBENCH, a framework that separates them. It builds a dependency graph over a tool catalog directly from tool schemas, audits an existing benchmark's reference sets against an executable runtime and human-anchored judges, and generates new tasks whose reference set is a sampled subgraph of chosen topology. On StableToolBench, a third to a half of reference tools turn out to be unnecessary, and scoring against the necessary ones raises every agent's recall by about a quarter: much of the reported count-driven degradation is an annotation artifact. On generated tasks with clean references, difficulty is dominated by subgraph size, but at equal size sequential composition stays harder than fan-out or fan-in, and the penalty disappears once dependencies are written out explicitly. What makes tool use hard, then, is inferring dependencies that tool descriptions leave implicit, and the tools agents omit are precisely those the instruction fails to motivate, not those the runtime can do without. We release the framework, a cleaned StableToolBench subset and 2,180 topology-controlled tasks, and we map where the structural ranking transfers: to execution and to a second catalog only in coarse form.

1 INTRODUCTION

Tool-calling agents are evaluated on benchmarks whose tasks are described, when difficulty is described at all, by a count: the number of tools a task requires, the number of hops, the number of turns. ToolBench separates single- from multi-tool instructions (Qin et al., 2023), TaskBench stratifies by the number of nodes in a node/chain/DAG structure (Shen et al., 2024), ToolHop by hop count (Ye et al., 2025), and leaderboards routinely report that accuracy falls as tool lists grow. A count is a convenient summary, but it is not a measure of difficulty in the sense that other fields use the term: it does not say what makes one five-tool task harder than another five-tool task, it does not separate a property of the task from a property of the annotation it is scored against, and it cannot be manipulated to test a hypothesis. This paper argues that tool count conflates two effects, shows how to measure each, and reports what survives once they are separated.

The first effect is a property of the reference set rather than of the task. Recall-based tool-calling metrics score an agent against a published set of ground-truth tools. A tool-calling task usually admits several correct plans, and a reference set that lists a verification call or a redundant lookup is a legitimate annotation choice, but it is not minimal, and recall then penalizes every agent that executes a shorter correct plan. This is not a hypothetical concern. Benchmark validity has become a first-order problem in agent evaluation: a checklist study of agentic benchmarks finds task-setup or reward-design issues that shift measured performance by up to 100% relative (Zhu et al., 2025); an automated audit of 168 benchmarks flags major issues in 25.7% of tasks and shows that removing them reorders leaderboards (Wang et al., 2026a); and one of the most widely reported coding benchmarks has been

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

retired by a frontier developer over contamination and flawed tests (OpenAI, 2026). These audits target instructions, environments and graders. Non-minimal reference sets are a distinct failure mode, specific to set-recovery metrics, and to our knowledge unquantified: we show on StableToolBench (Guo et al., 2024) that a third to a half of reference tools are non-essential by human and LLM judgment, and that scoring against the necessary subset alone raises recall by $+0.24$ to $+0.26$ for every agent we test.

The second effect is a property of the task: how the required tools compose. Other disciplines have long measured the difficulty of a structured task by the shape of its dependency graph rather than by its size. Cyclomatic complexity measures a program by the control-flow graph, not the line count (McCabe, 1976); workflow patterns classify processes by sequence, split and join (van der Aalst et al., 2003); in automated planning, the tractability of a domain is governed by the topology of its causal graph, with forks, inverted forks and chains behaving differently at equal size (Bylander, 1994; Chen & Giménez, 2010); and network science describes complex systems through recurring motifs (Milo et al., 2002). Tool-calling has the same structure available, because tools have typed inputs and outputs and therefore a dependency graph, but its benchmarks have not used it as a measure: the graph is implicit, hand-specified, or used only to generate tasks. Once the reference set equals the required subgraph by construction, we can ask whether the subgraph's topology predicts agent success after controlling for its size, depth, width and the length of the instruction. It does: at equal size, sequential motifs (Chain, Hourglass) are markedly harder than stars and trees, whereas the raw difficulty of large fan-in trees is almost entirely a size effect. This is a claim about tool selection, and we bound it: under execution the motif ranking changes, on a second catalog only the coarse ordering transfers, and on a knowledge-graph substrate whose relations are written in the context the chain penalty inverts, which is what one expects if chain difficulty comes from inferring dependencies that tool descriptions leave implicit.

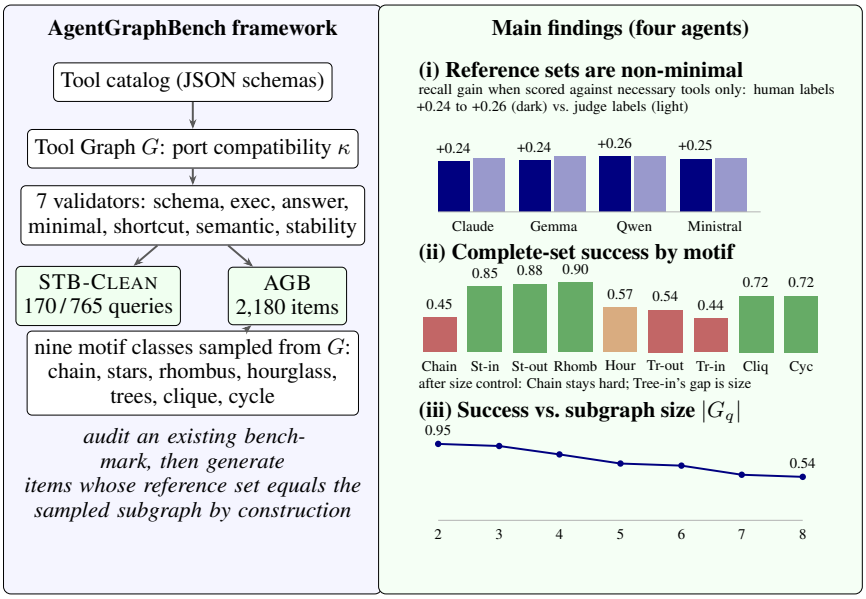


Figure 1: Graphical abstract. **Left:** AGB pipeline: tool catalog, Tool Graph, validation, yielding STB-CLEAN (170/765) and AGB (2,180 items, nine motif classes). **Right:** non-minimal reference sets (45–59% of StableToolBench reference tools judged non-essential by LLM judges, 34% by the human majority; recall $+0.24$ to $+0.26$ higher against the human-necessary subset), size effect (complete-set success 0.95 at $|G_q|$=2, 0.53 at $|G_q|$=8), and the motif effect that survives size control (Chain hardest at equal size; half of the Hourglass penalty is instruction coverage).

AGENTGRAPHBENCH makes both effects measurable on the same substrate. It builds a Tool Graph from a JSON catalog by port-level compatibility, audits an existing benchmark against an executable runtime, and generates items by back-instruction over subgraphs sampled for one of nine motif classes. Graph-conditioned generation is inherited from TaskBench; the increment is the automatically built graph, the finer taxonomy, the execution-grounded audit, and the quantification of reference-set

non-minimality with human-anchored judges. Applied to StableToolBench the framework yields STB-CLEAN (170 of 765 queries) and AGB (2,180 validated items, 451 with no answer-removable tool).

Our contributions, each with its scope (Figure 1):

- **Reference sets are non-minimal, and this changes what recall measures.** On StableToolBench, three LLM judges anchored by a six-annotator study mark 45–59% of reference tools non-essential (34% under the strict human majority); scoring against the human-necessary subset raises recall by $+0.24$ to $+0.26$ for each of four agents. Scope: one benchmark, judgments without plan context.
- **Topology is a difficulty axis that survives size control.** With reference sets equal to the generated subgraph, size dominates (complete-set success 0.95 at $|G_q|{=}2$, 0.53 at $|G_q|{=}8$) and motif remains a significant predictor after controlling for size, depth, width and question length; a chain-vs-DAG split recovers about 70% of the motif effect and the nine-way taxonomy adds a smaller, significant increment. Scope: tool selection with implicit dependencies.
- **Where the axis transfers.** The size gradient and the hardness of fan-in trees transfer to a second catalog and to a knowledge-graph substrate whose relations are written in the context; under execution on the same items the motif ranking changes; the nine-way ranking transfers to none of the three. Each transfer test is reported with its own scope statement.
- **A framework and two released sets.** A Tool Graph constructor, a seven-validator library of which three decide the released sets, motif samplers and a provider-agnostic agent harness, with every reported number rendered by a committed script.

1.1 RELATED WORK

API-Bank (Li et al., 2023), ToolBench (Qin et al., 2023), Gorilla and BFCL (Patil et al., 2024; 2025), ToolSandbox and τ-bench (Lu et al., 2025; Yao et al., 2024) cover runnable dialogues, thousands of real APIs, function-call correctness and stateful evaluation, and where they stratify difficulty they do so by count: single- vs. multi-tool, number of calls, hop depth, distractor count. NESTFUL (Basu et al., 2024) and ComplexFuncBench (Zhong et al., 2025) isolate nested and long-horizon call sequences, ToolHop (Ye et al., 2025) builds multi-hop tasks with verifiable answers, and ToolLoad-Bench (Wang et al., 2026b) raises a parametric "cognitive load" by inserting edges into a tool interaction graph; none separates the reference set from the task or asks whether the shape of the required subgraph, rather than its size, predicts failure. Generating tasks from the subgraphs of a tool graph is established: TaskBench (Shen et al., 2024) introduced a Tool Graph with resource-type edges, samples node, chain and DAG subgraphs of one to ten tools and writes requests by back-instruction, the recipe our generator follows; Magnet (Yin et al., 2025) turns signature-graph paths into multi-turn trajectories, DICE-Bench (Jang et al., 2025) synthesizes dialogues over a curated dependency graph, ToolNet (Liu et al., 2024) organizes tools as a transition graph for navigation, WorfBench (Qiao et al., 2025) scores workflows against graph-structured references, and MCP-Bench and ComplexMCP (Wang et al., 2025; Li et al., 2026) evaluate against live interdependent servers (Table 5, Appendix B). Our increment over TaskBench is the automatically built port-level graph, a nine-motif taxonomy in place of node/chain/DAG, an executable-runtime audit and the quantification of reference-set non-minimality; that a chain-vs-DAG split already recovers most of the topology effect is reported as a calibration of that increment, not hidden by it. We work on StableToolBench (Guo et al., 2024) because it couples reference annotations with an executable runtime.

Label errors and artifacts in benchmarks are well documented, from annotation artifacts in NLI (Gururangan et al., 2018) and label errors in ten standard test sets (Northcutt et al., 2021) to errors in 6.5% of MMLU items (Gema et al., 2024) and "platinum" subsets curated to remove noise (Vendrow et al., 2025). For agents, BetterBench (Reuel et al., 2024) aggregates construction criteria, the Agentic Benchmark Checklist (Zhu et al., 2025) formalizes task and outcome validity and finds flaws in most tool-providing benchmarks, ABA (Wang et al., 2026a) and BenchGuard (Tu et al., 2026) automate task-level auditing with agentic LLMs, and construct validity has been raised as the missing frame for LLM evaluation (Salaudeen et al., 2025). These audits classify issues as instruction, environment or grader defects; a non-minimal reference set is none of these: the instruction is clear, the environment works and the grader is correct, but the set it compares against penalizes valid plans. Our audit

targets that failure mode with an executable runtime and human-anchored judges, STB-CLEAN is the analogue of a platinum subset for a recall-scored tool benchmark, and LLM-as-judge methodology (Zheng et al., 2023; Liu et al., 2023) and the alternative annotator test (Calderon et al., 2025) supply the validation protocol.

That difficulty lives in dependency structure rather than in size predates LLMs: cyclomatic complexity measures a program by its control-flow graph (McCabe, 1976), workflow patterns classify processes by sequence, split, synchronization and choice (van der Aalst et al., 2003), and organizational psychology defines task complexity by component, coordinative and dynamic structure rather than by step count (Wood, 1986). In automated planning the complexity of a domain is governed by its causal graph (Bylander, 1994), and chains, forks, inverted forks and bounded treewidth make planning tractable or intractable at the same instance size (Bäckström & Nebel, 1995; Helmert, 2006; Chen & Giménez, 2010); network science describes systems by over-represented motifs (Milo et al., 2002), and parameterized complexity measures hardness by a structural parameter at fixed input size (Downey & Fellows, 1999). Our taxonomy and the depth/width controls of Section 5.3 apply this perspective to tool-calling; star-in and star-out are the inverted-fork and fork causal graphs of planning, whose prediction that fan-in is the hard case is what we observe for trees before size control. For LLMs, Dziri et al. (2023) predict failure from the depth and width of a computation graph, compound divergence measures compositional generalization independently of length (Keysers et al., 2020), MuSiQue composes multi-hop questions along explicit reasoning graphs (Trivedi et al., 2022), NLGraph and GraphArena test reasoning over graphs given in the context (Wang et al., 2023; Tang et al., 2024), PlanBench measures planning by plan length and domain (Valmeekam et al., 2023), and item difficulty has been estimated post hoc by item response theory (Rodriguez et al., 2021; Vania et al., 2021) or training dynamics (Swayamdipta et al., 2020). Our measure is a priori and manipulable: motif and size are chosen at sampling time, which allows the controlled regression of Section 5.3 and the transfer tests of Section 6, and the knowledge-graph substrate of Appendix O connects our findings to the in-context graph-reasoning setting of these benchmarks.

2 PROBLEM STATEMENT AND TOOL GRAPH

Let $\mathcal{T} = \{t_1, \ldots, t_n\}$ be a tool catalog specified only by a JSON description; each tool t is a tuple $(I_t, O_t, \mathrm{name}_t, \mathrm{desc}_t)$ of input ports, output ports, name and description, and each port carries a name, a declared type, a description and a required flag. A *port-level compatibility function* $\kappa(o, i) \in [0, 1]$ scores every output port $o \in O_u$ against every input port $i \in I_v$, $u, v \in \mathcal{T}$; with a threshold τ the port-level edges $E_P = \{(o, i) \mid \kappa(o, i) \geq \tau\}$ induce the directed Tool Graph $G = (V, E)$, $V = \mathcal{T}$, with $(u, v) \in E$ iff some $(o, i) \in E_P$ joins u to v.

Tool Graph construction. G is built automatically from the JSON catalog. *Schema normalization*: for every tool t we extract I_t and O_t, normalize field names (e.g., lat→latitude), map raw data types to a common type system, and flatten nested output objects into atomic fields (e.g., `rgb.red`). *Compatibility scoring*: $\kappa(o, i)$ combines type compatibility, lexical and embedding similarity of port names and descriptions, tool-level description similarity, and a penalty for generic outputs such as `result` or `code`; each edge with $\kappa(o, i) \geq \tau$ is labelled with its provenance, and the port correspondences and confidence scores behind every tool-level edge are stored for downstream use (Appendix C).

For a question q the relevant object is a *task subgraph* $G_q = (V_q, E_q) \subseteq G$ sufficient to derive the target answer. It is *executable* if a plan $\pi_q = (t^{(1)}, \ldots, t^{(m)})$ over V_q satisfies all required inputs and yields the correct final answer. A question generally admits several such plans, and a benchmark publishes one of them as its *reference set* $R_q \subseteq \mathcal{T}$, against which an agent's selection $S_q \subseteq \mathcal{T}$ is scored by recall $|S_q \cap R_q|/|R_q|$.

Reference sets and minimality. We call R_q *non-minimal* if it contains a tool t such that some valid plan over $R_q \setminus \{t\}$ still yields the correct answer. Non-minimality is not an error in the annotation (the extra tool may be a verification step or an alternative producer of the same value), but it changes what recall measures: an agent that executes a shorter correct plan is penalized for every tool it did not need, whereas execution success is indifferent to which valid plan was chosen. Auditing a benchmark therefore means measuring, for each $t \in R_q$, whether t is necessary; generating one

means constructing items whose reference set equals the generated subgraph by construction, with the necessity of each tool then measured by a structural and an answer-grounded test (Section 3).

3 VALIDATION AND GENERATION

The framework defines seven validators; the sets released in this paper are decided by three of them, V_{exec}, V_{sem} and V_{min}, as stated per set in Section 5.2, and the other four are implemented but not applied to the reported sets (Table 12). The overall decision is the binary function $\mathcal{V}(x_q, G_q, \pi_q, a_q) = V_{\text{schema}} \wedge V_{\text{exec}} \wedge V_{\text{answer}} \wedge V_{\text{minimal}} \wedge V_{\text{shortcut}} \wedge V_{\text{semantic}} \wedge V_{\text{stability}}$ (Figure 2).

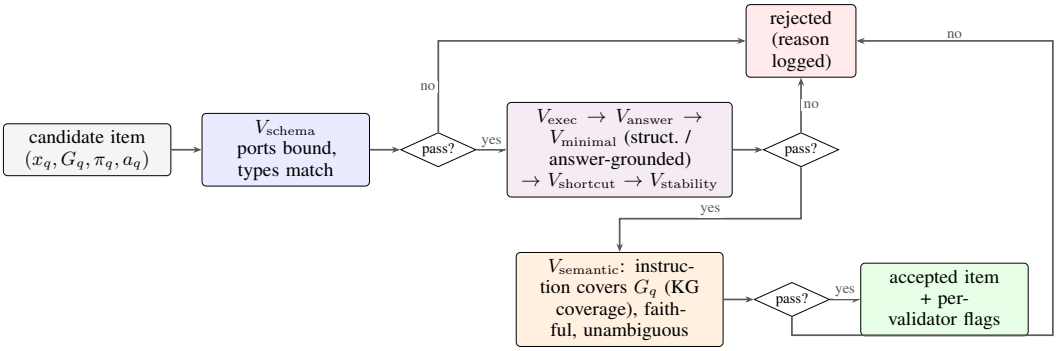


Figure 2: Overview of the validation pipeline combining formal checks, graph-based validation (when an executable tool environment is available), and LLM-based semantic validation.

V_{schema} **(structural).** Verifies that all source inputs $S_q = \{i \in I_t \mid t \in V_q,\ i \text{ is unsatisfied by upstream output}\}$ can be bound from the question, no required input is unbound, and the requested output aligns with the designated target port y_q.

V_{exec} **(execution).** Executes π_q step-by-step in the tool environment, propagating intermediate bindings between tools. $V_{\text{exec}} = 1$ iff all calls succeed without missing arguments and the final output is produced.

V_{answer} **(answer consistency).** Compares the executed answer $\hat{a}_q$ against the expected a_q via $V_{\text{answer}} = \mathbf{1}[\Delta(\hat{a}_q, a_q) \leq \varepsilon]$, where Δ is an application-specific equivalence relation.

V_{minimal} **(minimality).** Tests via ablation that no proper subgraph of G_q suffices. We distinguish two notions and never conflate them: *structural redundancy* ($V_{\text{min}}^{\text{struct}}$) flags a tool whose every output port is also produced by another tool in G_q (a pure-topology test that is satisfied by construction in Star-in and Clique motifs) and *answer-grounded minimality* ($V_{\text{min}}^{\text{answer}}$) flags a tool whose removal leaves execution successful and the sink answer unchanged. A third, judge-based proxy (V_{min}^{*}, Table 9) asks an LLM whether any listed tool is unnecessary for the instruction.

V_{shortcut} **(shortcut) and** $V_{\text{stability}}$ **(stability).** V_{shortcut} searches for shorter alternative plans that solve x_q outside the intended equivalence class and rejects or reassigns the candidate if one is found; $V_{\text{stability}}$ requires, for stochastic or stateful tools, that repeated executions $\text{Exec}^{(1)}, \ldots, \text{Exec}^{(r)}$ agree on success and answer within an admissible equivalence class.

V_{semantic} **(semantic faithfulness).** Builds a local knowledge graph $K(x_q)$ from the instruction text, maps its entities onto $V_q \cup S_q$ and its relations onto E_q, and passes the instruction when entity coverage is at least θ_{cov}, the share of unmapped entities at most θ_{red} and relation agreement at least θ_{edge} (definitions in Appendix C).

Generation. Given a sampled subgraph G_q, the goal is a natural-language question x_q whose answer requires composing the tools in G_q. For each subgraph we (i) designate a target output port $y_q \in O_{t^*}$, (ii) identify the source inputs S_q that must be provided externally, (iii) construct an

execution plan π_q by topological ordering (for the two cyclic motifs, Cycle and Clique, the plan is the spanning path fixed at sampling time; Appendix E), (iv) instantiate values for S_q, and (v) execute π_q to obtain a_q and the intermediate bindings B_q. Subgraph, source inputs, target output and execution result are converted into a structured task specification $\sigma_q = (\text{goal}_q, \text{facts}_q, \text{constraints}_q)$, the LLM is prompted with σ_q for k candidate questions, and the candidate with the highest aggregate quality score (naturalness, faithfulness, clarity) is kept. The item then passes through $\mathcal{V}$; V_{exec} decides membership in AGB and the LLM-judged validators are carried as per-item flags (Section 5.2).

4 Experimental Setup

Datasets. We use **StableToolBench** (Guo et al., 2024) both as the benchmark to be audited and as the source tool catalog from which AGB items are synthesized. Three nested sets are used throughout: *STB*, the full 765-query benchmark as published; STB-Exec, the 560 queries whose GT tools are reachable in the current `/virtual` deployment (those passing V_{exec} alone; the executability-cleaning baseline); and STB-Clean, the 170 queries that also pass V_{sem} and the judge-based minimality proxy V^*_{min} (per-query flags for all 765 queries are released; Section 5.2, Table 12). The AGB set comprises 2,180 items spanning nine motif classes (chain, star-in, star-out, rhombus, hourglass, tree-in, tree-out, clique, cycle), produced by sampling subgraphs from the StableToolBench tool catalog and synthesizing instructions for each. Each item's reference set is the sampled subgraph itself, so the non-minimality that affects existing benchmarks cannot confound the structural analysis (Section 3); structural redundancy within the subgraph is reported per motif in Appendix H. Instructions were generated between 30 April and 5 May 2026 in batches: Mistral Small 3.2 24B (via OpenRouter and the Mistral API) wrote the Chain, Star, Rhombus, Tree and smaller Clique items ($|G_q| \leq 7$; 1,681 validated items), Claude Haiku 4.5 the Cycle, Hourglass and larger Clique items (499), so generator and motif are partly confounded (Section 6); execution validation ran on 4–6 May 2026.

LLM judges and human study. For tool-necessity classification we use three independent judge models, Claude Haiku 4.5, GPT-5-Mini, and Qwen3-235B-A22B (OpenRouter slugs `anthropic/claude-haiku-4.5`, `openai/gpt-5-mini`, `qwen/qwen3-235b-a22b`; temperature 0; queried in May and September 2026), each receiving *(query, tool name, tool description)* and labelling the tool NECESSARY/OPTIONAL/UNNECESSARY with a one-sentence justification (prompt in Appendix D). To validate the LLM judges, six human annotators re-label a stratified 200-pair sample (small/medium/large GT sets), with three labels per pair via deterministic round-robin assignment. The same prompt is used for humans and LLMs.

Audit and cross-agent harness. The audit (Section 5.1) reports raw and adjusted recall of the four panel agents on *STB* and STB-Exec, adjusted against the three-judge consensus and, separately, the human strict majority. The cross-agent evaluation (Section 5.3) runs four agents from distinct model families, Claude Haiku 4.5, Gemma-4-31B-IT, Qwen3.6-Flash, Ministral-14B (OpenRouter slugs `anthropic/claude-haiku-4.5`, `google/gemma-4-31b-it`, `qwen/qwen3.6-flash`, `mistralai/ministral-14b-2512`; temperature 0; all runs on 6 May 2026), on *STB*, STB-Exec, and AGB, in a tool-selection setup with a 25-tool candidate list per item (GT plus deterministically-shuffled noise), which isolates tool selection from retrieval and runtime stability. The Qwen3.6-Flash endpoint does not expose `tool_choice`, so that agent selects tools from a plain-text listing while the other three receive the candidates as function-calling tools; its numbers are therefore not strictly comparable at the harness level, and we flag this wherever it matters.

Metrics and uncertainty. We report the necessity distribution (% per category), raw recall $|S \cap GT|/|GT|$, adjusted recall $|S \cap N|/|N|$ with $N \subseteq GT$ the necessary subset, complete-set success ($GT \subseteq S$) and Cohen's κ between labelers. Every recall, success or agreement cell is a mean over items printed as mean $\pm$ the half-width of its 95% percentile-bootstrap CI (B=10,000, fixed seed; resampling units in the Reproducibility statement, exact bounds in Appendix J); regression coefficients carry cluster-robust CIs. A CI that excludes zero stands in for a significance claim.

5 EXPERIMENTAL RESULTS

We report the audit of StableToolBench (Section 5.1), the generated and cleaned sets (Section 5.2) and the motif and size analysis on AGB (Section 5.3). Structural quantities use the definitions of Appendix E.

5.1 REFERENCE SETS ARE NON-MINIMAL, AND THE GAP IS REPRODUCIBLE

Necessity labels. Three LLM judges label each reference tool of the 200 human-study (query, tool) pairs as NECESSARY, OPTIONAL or UNNECESSARY; six annotators label the same pairs, three votes each (Appendix F, Table 8). The judges mark 45–59% of reference tools non-essential (OPTIONAL or UNNECESSARY), the strict human majority 34% on the 189 pairs with a majority. Judge–judge κ is .55–.56 on those pairs and judge–human κ .54–.65, above the inter-human κ of .44 (Table 17). Under the alternative annotator test (Calderon et al., 2025) only Claude Haiku 4.5 passes at ε=0.10 (Qwen3-235B at 0.15, GPT-5-Mini at 0.20), so we rely on the three-judge consensus anchored by the human study rather than on any single judge, and we keep the context-free prompt because showing the judge the full reference set lowers agreement with the humans ($\Delta\kappa = -.12$; Appendix K).

Adjusted recall. Restricting recall to the tools judged necessary raises it for every one of the four agents of Section 5.3: on STB raw recall .52–.54 becomes .79 ($\Delta \approx +.26$, every CI above zero; Table 2 in Appendix A), and the gap barely narrows on STB-EXEC, so unreachable tools are not its cause. It survives human-only labels: with the annotators' strict-majority necessary set, Δ is $+.24$ to $+.26$ for every agent (95% CIs exclude $+.18$; the judge definition gives $+.25$ to $+.29$ on the same pairs; Table 18). A raw-recall leaderboard therefore mainly measures over-selection of redundant reference tools, not omission of necessary ones.

5.2 GENERATING AND CLEANING BENCHMARKS

Generation acceptance. Sampling and generation across motifs yield 3,032 instructions, of which 2,180 pass V_{exec} on the runtime and form AGB (Table 9); every sampled subgraph received an instruction, so all attrition is at V_{exec} (e.g. Rhombus $300 \to 142$). V_{sem} is $\geq .96$ on Chain, the stars and Rhombus and .46 on Hourglass and Cycle; the judge-based proxy $V^*_{\min}$ is 1.00 on the stars and Rhombus but .05 on Hourglass and .21–.23 on the trees. Structural redundancy (Table 10) is a property of the motif, not an annotation error: 0 for Chain, Star-out, Tree-out and Cycle, near or at 1 for Star-in (.96), Tree-in (1.00) and Clique (.85), where several tools feed the same port by construction.

Cleaning StableToolBench. On the 765 queries, V_{exec} retains 73.2%, V_{sem} 81.8% and $V^*_{\min}$ 26.8%; their conjunction, STB-CLEAN, keeps 170 queries (22.2%; Table 12). STB-CLEAN is a diagnostic slice, not a drop-in replacement: its size depends on the current `/virtual` deployment and on two LLM-judged validators, and $V^*_{\min}$ removes non-minimal rather than wrong reference sets, so we release per-query flags for all 765 queries.

Answer-grounded minimality. An AGB reference set equals the sampled subgraph, so no tool was added by annotation; whether each tool is needed for the answer is measured. Every item ships with its plan, the sink answer recorded at validation and its validator flags. Re-executing each item without one reference tool at a time (original arguments replayed, sinks kept; Table 11) finds an answer-removable tool in 79% of items, fewest where leaves consume a hub's output (Hourglass 35%, Star-out 58%) and most (82–90%) where intermediate values are stated in the instruction or not consumed by the simulated runtime. The 451 items (21%) without such a tool form AGB-STRICT; the conclusions below hold on that subset (Appendix M).

5.3 TOOL GRAPH MOTIFS AS AN AXIS OF CONTROLLED STRUCTURAL DIFFICULTY

We evaluate the four agents on STB, STB-EXEC and AGB under the selection-only harness of Section 4. On AGB $\Delta \equiv 0$ by construction and raw recall is 0.87–0.93 (Table 2), so the analyses below isolate structural effects from annotation noise.

Topology drives difficulty. Slicing AGB success by motif and by $|G_q| \in \{2, \ldots, 8\}$ (Tables 3 and 4, Appendix A) gives two effects that hold for each agent. Rhombus, Star-out and Star-in are easiest (per-agent success 0.77–0.98), Tree-in and Chain hardest (0.37–0.51), with Hourglass, Tree-out, Clique and Cycle in between; and success degrades monotonically with size, from ≥ 0.93 at $|G_q|$=2 to ≤ 0.60 at $|G_q|$=8.

Controlling for size and surface properties. Motif and size are confounded by construction (Tree-in items have 9.3 reference tools on average, Rhombus 4.0), so we fit a logistic regression of per-item success on motif with $|G_q|$, longest-path depth, BFS width, $\log$ question length and agent fixed effects as controls (Table 1; cluster-robust CIs by item). Motif remains highly significant after the controls (LRT χ^2_8=595.7, $p \approx 10^{-123}$; McFadden R^2 .218 $\rightarrow$ .272), but the effect is concentrated in sequential motifs: at equal size Chain (-1.48 log-odds vs. Star-out) and Hourglass (-1.20) are markedly harder, whereas Tree-in (-0.31, 95% CI $[-0.73, +0.10]$) is indistinguishable from Star-out; about 90% of its raw gap to Rhombus is size and question length. Omissions track the instruction, not the runtime: judge-flagged tools are omitted 61–64% of the time, the rest 6–13%, and answer-removable tools no more often than necessary ones; with the judge-flag count as a covariate motif stays significant, the Hourglass penalty halves ($-1.20 \rightarrow -0.55$) and Chain keeps most of its own ($-1.48 \rightarrow -1.12$; Tables 25–26). Size is the dominant single predictor (-0.70 log-odds per additional tool) and longer questions help ($+1.57$ per log-unit of words). A TaskBench-style chain-vs-DAG split already recovers about 70% of the motif R^2 gain; the nine-way taxonomy adds a significant but smaller increment (χ^2_7=172.8, $p \approx 10^{-34}$, lower AIC), carried by the Hourglass penalty and the Tree-out/Cycle advantage. Motif effects differ across agents in magnitude (motif$\times$agent interaction p=6×10^{-9}), so we claim a shared pattern, not identical effect sizes. Candidate-tool difficulty is held fixed by the seeded 25-tool lists; size-matched and question-length-stratified slices are in Appendix M.

Table 1: Logistic regression of per-item success (all reference tools selected) on motif, controlling for $|G_q|$, longest-path depth, BFS width, $\log$ question length (words) and agent fixed effects; 8,646 (agent, item) rows over 2,180 validated AGB items and 4 agents. Motif coefficients are log-odds relative to Star-out, with 95% cluster-robust CIs (clustered by item). *Raw* is the unadjusted success rate. Likelihood-ratio tests compare nested fits.

Motif	**Raw success**	**Adjusted log-odds vs. Star-out**	p
Star-out	.879	(baseline)	—
Star-in	.846	-.36 $\pm$.38	.059
Rhombus	.899	-.08 $\pm$.52	.758
Hourglass	.571	-1.20 $\pm$.53	$<$.001
Chain	.447	-1.48 $\pm$.54	$<$.001
Tree-out	.543	+.57 $\pm$.58	.055
Tree-in	.439	-.31 $\pm$.42	.140
Clique	.724	-.28 $\pm$.60	.351
Cycle	.723	+.60 $\pm$.61	.052
$\lvert G_q \rvert$ (per tool)	—	-.70 $\pm$.14	$<$.001
depth (per level)	—	-.05 $\pm$.12	.383
width (per vertex)	—	+.19 $\pm$.11	$<$.001
log question length	—	+1.57 $\pm$.32	$<$.001
LRT motif (M1 vs. controls-only M0): $\chi^2_8 = 595.7$, $p < .001$; McFadden R^2 0.218 $\rightarrow$ 0.272			
LRT motif with $\lvert G_q \rvert$ as a factor: $\chi^2_8 = 695.3$, $p < .001$			
LRT motif$\times$agent interaction (M2 vs. M1): $\chi^2_{24} = 86.4$, $p < .001$			
TaskBench collapse (Chain vs. DAG; Node is empty for AGB): Chain log-odds -1.71 $\pm$.25; LRT vs. M0 $\chi^2_1 = 422.9$			
LRT nine motifs vs. TaskBench classes (M1 vs. M_{tb}): $\chi^2_7 = 172.8$, $p < .001$			
AIC 8666 (M0) / 8246 (M_{tb}) / 8087 (M1); McFadden R^2 0.256 (M_{tb})			

Metric robustness. Precision is .98–.99 in every cell and agents select fewer tools than the reference, so low success on long motifs is omission, not over-selection (Appendix L). Exact-set match ranks motifs identically to success ($\rho \geq .98$); F1, which gives partial credit, compresses the motif spread from $\approx .5$ to $\approx .1$. The agent ranking depends on the metric, so we make no single-agent-ordering claim.

Execution-grounded evaluation. On 50 validated items per motif the four agents also filled arguments, called the runtime, read the responses and answered (Appendix P, Table 34). Selection-only success of .63–.70 becomes clean execution on .37–.45 of the items (.08 for Ministral-14B, which fails the action protocol on 19% of items) and a structurally correct sink answer on .26–.30. The pre-registered test of whether the motif ranking survives the switch fails for every agent (Spearman ρ .11–.65): under execution Hourglass is uniformly hardest and Cycle easiest, while Chain and Tree-in are mid-ranked. The topology claims of this section are claims about tool selection; a knowledge-graph substrate with explicit relations gives a third boundary (Appendix O), discussed with the other two in Section 6.

6 Discussion and Limitations

Where the ranking holds. Three transfer tests bound the claim. Under execution on the same items (Appendix P) Hourglass is uniformly hardest and Cycle easiest: execution rewards bottlenecks and few sinks, not exact recovery. On a second, unrelated catalog (ToolLinkOS, 573 tools; audit-only, no runtime) the size gradient and the hardness of Tree-in and Chain transfer for every agent, but the nine-way ranking does not (Spearman ρ .41–.79 vs. AGB), partly because the catalog has only 10 rhombi and no directed cycles longer than two (Appendix N). On a knowledge graph whose relations are written in the context (Appendix O) Tree-in stays hardest and hardens with size, but Chain becomes the easiest motif, as expected if chain difficulty comes from inferring dependencies that tool descriptions leave implicit. What holds everywhere, execution included (Table 35), is that size dominates; fan-in trees are hard on both selection substrates; the sequential penalty at equal size is a tool-selection finding for settings in which the dependency structure is not given.

Limitations. (i) V_{exec} and the answer-grounded tests depend on a simulated runtime that answers even on empty inputs, so answer-necessity is a weak test (79% of items have an answer-removable tool) and structural redundancy, reported separately, is near 1 by construction for fan-in motifs and Clique. (ii) On the 600 tree items and 97 early chain items without port-level edges the structural test falls back to tool-level redundancy and is conservative. (iii) Adjusted recall depends on the judge consensus; raw and adjusted are always reported side by side. (iv) The 25-tool candidate list removes retrieval noise, so absolute recall is an upper bound on an open-catalog regime. (v) Motif and surface properties are coupled by generation; the regression controls for size, depth, width and question length and reassigns Tree-in's raw difficulty to size, but a study that matches surface properties across motifs remains open. (vi) Annotators and the judge prompt both lack the query's full plan; a plan-aware judge is stricter and agrees less with them (Appendix K). (vii) The motif ranking differs under execution (Appendix P) and transfers only in part to a knowledge graph with explicit relations (Appendix O); the topology claims are about tool selection where dependencies must be inferred. (viii) Four agents from four model families are evaluated; the shared pattern, not the effect sizes, is what we claim transfers. (ix) Instruction generation was batched by motif (Section 4): Cycle, Hourglass and the larger Clique items were written by Claude Haiku 4.5, the rest by Mistral Small 3.2, so the Hourglass and Cycle coefficients of Table 1 are not separable from the generator and their lower V_{sem} coincides with the generator boundary; Chain and its comparison motifs share a generator, and Claude Haiku 4.5 is generator, judge and agent for that slice. (x) Of the seven validators the library defines, the released sets are decided by V_{exec}, V_{sem} and $V_{\min}$ (Table 12); V_{schema}, V_{answer}, V_{shortcut} and $V_{\text{stability}}$ are implemented but not applied to the sets we report.

7 Conclusion

Tool count conflates a property of the annotation with a property of the task. On StableToolBench a third to a half of reference tools are non-essential, and scoring against the tools a human majority marks necessary raises recall by $+0.24$ to $+0.26$ for every agent. Once reference sets equal the generated subgraph, size dominates (complete-set success 0.95 at $|G_q|{=}2$ to 0.53 at $|G_q|{=}8$) and topology still matters at equal size, with Chain the robust hard motif and half of the Hourglass penalty traced to instructions that do not motivate every joining tool. These are claims about tool selection: under execution the motif ranking changes, and only the coarse ordering transfers to a second catalog. AgentGraphBench makes both effects measurable and is released with STB-Clean (170 of 765 queries), AGB (2,180 items; 451 with no answer-removable tool) and every rendered number.

AI USE STATEMENT

In this work we used generative AI tools for the following tasks with required disclosure: (i) *code*: an LLM coding assistant (Claude Code) wrote or refactored parts of the analysis and rendering scripts, the test suite and the experiment reports in the released repository; every script that produces a number in this paper was run by the authors and its output is committed with the code; (ii) *writing*: the same assistant drafted and edited passages of the abstract, introduction, related work, appendices and these statements, which the authors revised; (iii) *data*: LLMs are part of the studied method: they generate benchmark instructions (Section 3), act as tool-calling agents and act as necessity judges (Section 4); every such use is described in the paper with model identifiers and dates. We did not use generative AI to produce or alter experimental results, human-annotation labels, or citations; all references were verified against their venue or arXiv pages by the authors. We have reviewed all AI-assisted work and take responsibility for the final content of this work, including text, claims and artifacts produced with the aid of generative AI.

ETHICS STATEMENT

The human study (Section 4) asked six annotators to label 200 (query, tool) pairs drawn from a public benchmark. The pairs contain no personal data; annotators received the written instructions reproduced in Appendix D, could stop at any time, and are identified in the released data only by an index. The annotators are salaried members of the authors' laboratory and received no additional compensation; several are authors of this paper. The study collects judgments about tool descriptions from a public benchmark and involves no personal data, deception or risk, so it falls outside the scope of ethics-board review at the authors' institution. The released datasets (STB-CLEAN, AGB) are derived from StableToolBench, whose tool catalog and virtual API server we use under their Apache-2.0 licence; our code is released under the MIT licence and our data under CC BY 4.0. The benchmark contains synthetic instructions over public API descriptions; we found no personal or sensitive content, but the underlying catalog is third-party and we did not audit every tool description. LLM judges introduce model-specific bias into the necessity labels; we quantify it against human annotators rather than assume it away, and we report the negative results of that comparison.

REPRODUCIBILITY STATEMENT

All numbers in this paper are produced by scripts in the released repository: one render script per table and figure (`scripts/render/`), reading committed distilled data (validated items, agent runs, judge labels, annotator files) and writing the exact LaTeX tables and JSON results that the paper inputs. Bootstrap confidence intervals use a fixed seed and $B{=}10{,}000$. The Tool Graph construction, validators, samplers, generation pipeline and agent harness are the open-source library described in Appendix C; model identifiers, prompts (Appendix D) and run dates are given in Section 4. Steps that require the StableToolBench runtime or paid API calls are marked as such; every other result can be re-rendered on a laptop with no keys. The code, the distilled data, every rendered table and a Croissant dataset card with Responsible-AI fields are released in an anonymized repository at `https://anonymous.4open.science/r/AgentGraphBench-E270`; the raw generation outputs are regenerable from the committed configurations.

Resampling units. The bootstrap resamples the benchmark record for recall and success-rate tables and the (query, tool) pair for judge agreement; within-item contrasts ($\Delta = \text{Adj} - \text{Raw}$ on the same record, $\Delta\kappa$ between two judges on the same 189 human-study pairs) use the paired bootstrap. Each AGB record is a unique sampled subgraph, so item-level resampling is also cluster-level resampling; the human-study set has $200/175{=}1.14$ pairs per query on average (max 3), so pairs are resampled.

Compute. The selection panel comprises 17,428 calls (4 agents $\times$ 4,357 items; 71.6 cumulative hours); the execution panel 12,038 LLM calls on 450 items per agent (US$24 at list prices); the second-catalog panel 1,476 selection calls plus 738 generation and judge calls; the knowledge-graph panel $\approx$34,500 calls (4 agents $\times$ 4,310 items $\times$ 2; US$24); the necessity audit 1,764 judge calls (588 pairs $\times$ 3 judges) plus 600 on the 200 human-study pairs and 800 in two prompt ablations (US$0.59). Execution validation issued one `/virtual` call per tool per generated item ($\approx$18,600 calls). All LLM calls went through OpenRouter or the Mistral API; no model was trained or fine-tuned.

REFERENCES

Christer Bäckström and Bernhard Nebel. Complexity results for SAS^+ planning. *Computational Intelligence*, 11(4):625–655, 1995.

Kinjal Basu, Ibrahim Abdelaziz, Kelsey Bradford, Maxwell Crouse, Kiran Kate, Sadhana Kumaravel, Saurabh Goyal, Asim Munawar, Yara Rizk, Xin Wang, Luis Lastras, and Pavan Kapanipathi. NESTFUL: A benchmark for evaluating LLMs on nested sequences of API calls. *arXiv preprint arXiv:2409.03797*, 2024.

Tom Bylander. The computational complexity of propositional STRIPS planning. *Artificial Intelligence*, 69(1–2):165–204, 1994.

Nitay Calderon, Roi Reichart, and Rotem Dror. The alternative annotator test for LLM-as-a-judge: How to statistically justify replacing human annotators with LLMs. *arXiv preprint arXiv:2501.10970*, 2025.

Hubie Chen and Omer Giménez. Causal graphs and structurally restricted planning. *Journal of Computer and System Sciences*, 76(7):579–592, 2010.

Rodney G. Downey and Michael R. Fellows. *Parameterized Complexity*. Springer, 1999.

Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems 36*, 2023.

Aryo Pradipta Gema, Joshua Ong Jun Leang, Giwon Hong, Alessio Devoto, Alberto Carlo Maria Mancino, Rohit Saxena, Xuanli He, Yu Zhao, Xiaotang Du, Mohammad Reza Ghasemi Madani, Claire Barale, Robert McHardy, Joshua Harris, Jean Kaddour, Emile van Krieken, and Pasquale Minervini. Are we done with MMLU? *arXiv preprint arXiv:2406.04127*, 2024.

Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 11143–11156, 2024.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 107–112, 2018.

Malte Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26: 191–246, 2006.

Kyochul Jang, Donghyeon Lee, Kyusik Kim, Dongseok Heo, Taewhoo Lee, Woojeong Kim, and Bongwon Suh. DICE-BENCH: Evaluating the tool-use capabilities of large language models in multi-round, multi-party dialogues. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. arXiv:2506.22853.

Daniel Keysers, Nathanael Schärli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafiniak, Tibor Tihon, Dmitry Tsarkov, Xiao Wang, Marc van Zee, and Olivier Bousquet. Measuring compositional generalization: A comprehensive method on realistic data. In *International Conference on Learning Representations (ICLR)*, 2020.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 3102–3116, 2023.

Yuanyang Li, Xue Yang, Longyue Wang, Weihua Luo, and Hongyang Chen. ComplexMCP: Evaluation of LLM agents in dynamic, interdependent, and large-scale tool sandbox. *arXiv preprint arXiv:2605.10787*, 2026.

Xukun Liu, Zhiyuan Peng, Xiaoyuan Yi, Xing Xie, Lirong Xiang, Yuchen Liu, and Dongkuan Xu. ToolNet: Connecting large language models with massive tools via tool graph. *arXiv preprint arXiv:2403.00839*, 2024.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 2511–2522, 2023.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, et al. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 1160–1183, 2025.

Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4): 308–320, 1976.

Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. Network motifs: Simple building blocks of complex networks. *Science*, 298(5594):824–827, 2002.

Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. Pervasive label errors in test sets destabilize machine learning benchmarks. In *Advances in Neural Information Processing Systems 34, Datasets and Benchmarks Track*, 2021.

OpenAI. Why we no longer evaluate SWE-bench Verified. `https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/`, 2026.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37: 126544–126565, 2024.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Benchmarking agentic workflow generation. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.07869; introduces WorfBench.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. BetterBench: Assessing AI benchmarks, uncovering issues, and establishing best practices. In *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024. arXiv:2411.12990.

Pedro Rodriguez, Joe Barrow, Alexander Miserlis Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

Olawale Salaudeen, Anka Reuel, Ahmed Ahmed, Suhana Bedi, Zachary Robertson, Sudharsan Sundar, Ben Domingue, Angelina Wang, and Sanmi Koyejo. Measurement to meaning: A validity-centered framework for AI evaluation. *arXiv preprint arXiv:2505.10573*, 2025.

Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. Taskbench: Benchmarking large language models for task automation. *Advances in Neural Information Processing Systems*, 37:4540–4574, 2024.

Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A. Smith, and Yejin Choi. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

Jianheng Tang, Qifan Zhang, Yuhan Li, Nuo Chen, and Jia Li. GraphArena: Benchmarking large language models on graph computational problems. *arXiv preprint arXiv:2407.00379*, 2024.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

Xinming Tu, Tianze Wang, Yingzhou Lu, Kexin Huang, Yuanhao Qu, and Sara Mostafavi. BenchGuard: Who guards the benchmarks? automated auditing of LLM agent benchmarks. *arXiv preprint arXiv:2604.24955*, 2026.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, Bartek Kiepuszewski, and Alistair P. Barros. Workflow patterns. *Distributed and Parallel Databases*, 14(1):5–51, 2003.

Clara Vania, Phu Mon Htut, William Huang, Dhara Mungra, Richard Yuanzhe Pang, Jason Phang, Haokun Liu, Kyunghyun Cho, and Samuel R. Bowman. Comparing test sets with item response theory. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

Joshua Vendrow, Edward Vendrow, Sara Beery, and Aleksander Madry. Do large language model benchmarks test reliability? *arXiv preprint arXiv:2502.03461*, 2025.

Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems 36*, 2023. NLGraph.

Junlin Wang, Federico Bianchi, Shang Zhu, Fan Nie, Yongchan Kwon, Bhuwan Dhingra, and James Zou. Automated benchmark auditing for AI agents and large language models. *arXiv preprint arXiv:2605.26079*, 2026a.

Qihao Wang, Yue Hu, Mingzhe Lu, Jiayue Wu, Yanbing Liu, and Yuanmin Tang. Beyond accuracy: A cognitive load framework for mapping the capability boundaries of tool-use agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026b. arXiv:2601.20412; introduces ToolLoad-Bench.

Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers. *arXiv preprint arXiv:2508.20453*, 2025.

Robert E. Wood. Task complexity: Definition of the construct. *Organizational Behavior and Human Decision Processes*, 37(1):60–82, 1986.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Junjie Ye, Zhengyin Du, Xuesong Yao, Weijian Lin, Yufei Xu, Zehui Chen, Zaiyuan Wang, Sining Zhu, Zhiheng Xi, Siyu Yuan, Tao Gui, Qi Zhang, Xuanjing Huang, and Jiecao Chen. ToolHop: A query-driven benchmark for evaluating large language models in multi-hop tool use. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2501.02506.

Fan Yin, Zifeng Wang, I-Hung Hsu, Jun Yan, Ke Jiang, Yanfei Chen, Jindong Gu, Long T. Le, Kai-Wei Chang, Chen-Yu Lee, Hamid Palangi, and Tomas Pfister. Magnet: Multi-turn tool-use data synthesis and distillation via graph translation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2503.07826.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.

Lucen Zhong, Zhengxiao Du, Xiaohan Zhang, Haiyi Hu, and Jie Tang. ComplexFuncBench: Exploring multi-step and constrained function calling under long-context scenario. *arXiv preprint arXiv:2501.10132*, 2025.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*, 2025. Agentic Benchmark Checklist.

A Main result tables

Tables 2–4 are the cross-agent recall table and the per-motif and per-size success tables discussed in Section 5.3; every cell carries the half-width of its 95% bootstrap CI and the exact bounds are in Appendix J.

Table 2: Recall comparison across agents and benchmark variants. Raw recall is $|\text{Selected} \cap \text{GT}|/|\text{GT}|$; adjusted recall is the same metric restricted to the subset of GT tools judged NECESSARY by the LLM-judge consensus. Δ = Adj. Recall − Raw Recall (smaller is better; expected to shrink under cleaning and under generation). AGB is restricted to validated items. Cells are means over benchmark records with the half-width of the 95% percentile-bootstrap CI (B=10,000, records resampled; Δ uses the paired bootstrap over records with a non-empty necessary subset); exact bounds in Table 13.

Agent	Benchmark	Raw Recall	Adj. Recall	Δ
Claude Haiku 4.5	StableToolBench	.532 ±.020	.789 ±.019	+.258 ±.019
	STB-Exec	.563 ±.025	.811 ±.022	+.249 ±.023
	AGB-Generated	.870 ±.010	—	—
Gemma-4-31B-IT	StableToolBench	.524 ±.021	.786 ±.019	+.263 ±.020
	STB-Exec	.543 ±.025	.792 ±.023	+.251 ±.023
	AGB-Generated	.910 ±.008	—	—
Qwen3.6-Flash	StableToolBench	.529 ±.021	.792 ±.019	+.265 ±.019
	STB-Exec	.547 ±.025	.807 ±.022	+.260 ±.023
	AGB-Generated	.929 ±.005	—	—
Ministral-14B	StableToolBench	.535 ±.020	.790 ±.019	+.256 ±.019
	STB-Exec	.556 ±.024	.800 ±.022	+.245 ±.022
	AGB-Generated	.886 ±.009	—	—

B Prior graph-based tool benchmarks

Table 5 positions this work against the graph-based generators and interdependent-tool benchmarks discussed in Section 1.1.

Table 3: Per-motif success rate (all reference tools selected, raw recall = 1.0) on AGB (validated items; the two-tool *bridge* items are Chain with $|G_q|$=2). The last column is the macro-average over motifs. Cells are means over records with the half-width of the 95% percentile-bootstrap CI (B=10,000, records resampled within the cell; the macro column resamples within each motif). The *Average* row pools the four agents' items.

Agent	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	Clique	Cycle	Macro avg.
Claude Haiku 4.5	.411 ±.045	.811 ±.048	.834 ±.050	.866 ±.056	.622 ±.108	.514 ±.073	.373 ±.066	.704 ±.048	.697 ±.051	.648 ±.021
Gemma-4-31B-IT	.456 ±.048	.877 ±.044	.930 ±.033	.888 ±.056	.554 ±.108	.614 ±.076	.493 ±.068	.782 ±.043	.786 ±.043	.709 ±.021
Qwen3.6-Flash	.472 ±.048	.928 ±.034	.956 ±.028	.976 ±.028	.608 ±.108	.593 ±.073	.505 ±.066	.666 ±.050	.721 ±.050	.714 ±.020
Ministral-14B	.450 ±.045	.771 ±.052	.795 ±.050	.873 ±.053	.500 ±.108	.455 ±.073	.387 ±.066	.745 ±.047	.687 ±.051	.629 ±.022
Average	.447 ±.023	.846 ±.023	.879 ±.021	.899 ±.026	.571 ±.056	.543 ±.037	.439 ±.033	.724 ±.024	.723 ±.024	.675 ±.011

Table 4: Success rate (all reference tools selected, raw recall = 1.0) on AGB (validated items) as a function of subgraph size $|G_q|$. Cells are means over records with the half-width of the 95% percentile-bootstrap CI (B=10,000, records resampled within the cell). The *Average* row pools the four agents' items.

Agent	$\|G_q\|$=2	$\|G_q\|$=3	$\|G_q\|$=4	$\|G_q\|$=5	$\|G_q\|$=6	$\|G_q\|$=7	$\|G_q\|$=8
Claude Haiku 4.5	.950 ±.030	.909 ±.039	.788 ±.042	.690 ±.054	.622 ±.065	.526 ±.068	.473 ±.064
Gemma-4-31B-IT	.964 ±.028	.952 ±.026	.841 ±.040	.732 ±.051	.726 ±.058	.623 ±.066	.600 ±.064
Qwen3.6-Flash	.960 ±.027	.941 ±.030	.859 ±.037	.756 ±.051	.713 ±.062	.596 ±.066	.564 ±.067
Ministral-14B	.926 ±.037	.900 ±.039	.782 ±.042	.648 ±.056	.656 ±.062	.512 ±.066	.502 ±.067
Average	.950 ±.015	.925 ±.018	.816 ±.020	.706 ±.026	.679 ±.032	.564 ±.033	.535 ±.033

C Implementation Details

The AGENTGRAPHBENCH library is implemented in Python with provider-agnostic LLM integration via `pydantic_ai`. All validation steps are implemented as composable Pydantic models with optional callback hooks, enabling integration with arbitrary tool environments. Configuration is YAML-based with environment-variable overrides.

Compatibility score (Section 2). For each pair of tools (u, v), candidate edges are generated by comparing every output port $o \in O_u$ with every input port $i \in I_v$. A compatibility score is computed:

$$\kappa(o,i) = \alpha\, s_{\text{type}}(o,i) + \beta\, s_{\text{name}}(o,i) + \gamma\, s_{\text{desc}}(o,i) + \delta\, s_{\text{context}}(u,v) - \lambda\, s_{\text{generic}}(o),$$

where s_{type} measures type compatibility, s_{name} and s_{desc} measure lexical and semantic similarity (via sentence embeddings), s_{context} measures tool-level description similarity, and s_{generic} penalizes weakly informative outputs (e.g., `result`, `message`, `code`).

Threshold and provenance. A directed port-level edge is added whenever $\kappa(o,i) \geq \tau$. Each accepted edge is assigned a provenance label (*exact*, *normalized*, *alias*, *nested*, or *description-based*).

The graph as an object. Table 6 lists the builder configuration in the released code, the catalog it was applied to, and the part of the graph that the validated AGB items use. All four similarity terms carry unit weight and τ=1.5; the generic-output penalty was configured with λ=0, so it was inactive when the graph was built, and every edge that reached an item therefore has $\kappa \geq 1.8$ on name, type, description and context similarity alone. The catalog has 4,070 tools, of which 3,085 expose both input and output ports; the 2,180 items touch 931 tools through 4,428 distinct tool-level dependency edges, and the 1,707 items whose sampler kept port-level edges carry 3,455 distinct port edges, seven in ten of them with an exact port-name match. The edge count of the complete catalog graph is not reported: no committed artifact holds it, only the subgraphs that reached validation.

V_{semantic} metrics (Section 3). The validator builds a local knowledge graph $K(x_q) = (V_K, E_K)$ from the instruction text and aligns it with G_q. Three metrics are computed:

$$C_{\text{cov}}(x_q) = \frac{|\{v \in V_K : \Phi(v) \in V_q \cup S_q\}|}{|V_K|}, \quad C_{\text{red}}(x_q) = \frac{|\{v \in V_K : \Phi(v) \notin V_q \cup S_q\}|}{|V_K|},$$

Table 5: Graph-based tool-use benchmarks and generators. *Graph*: how the tool graph is obtained (from tool schemas automatically, curated, synthesized, or from usage traces); *classes*: named task-structure classes used to sample or stratify; *runtime*: tools are executed at evaluation time; *minimality*: reference tool sets are tested by ablation on the runtime; *audit*: applied to an existing benchmark's annotations; *answers*: items carry an executable reference answer; *GT noise*: annotation noise in reference tool sets is quantified. n/r = not reported. "partial" for AGB minimality: structural redundancy is reported per motif (Appendix H); the answer-grounded ablation is defined in Section 3.

System	Graph	Classes	Runtime	Minimality	Audit	Answers	GT noise
TaskBench	curated	node/chain/DAG	no	no	no	no (structure)	no
Magnet	from schemas	path ops	at synthesis	no	no	no	no
DICE-Bench	curated	rounds/parties	simulated	no	no	call match	no
ToolNet	from usage	none	n/a	no	no	no	no
ToolHop	no graph	hop depth	local code	no	no	yes	no
ToolLoad-Bench	synthesized	parametric load	n/r	no	no	n/r	no
MCP-Bench	no graph	none	live MCP	no	no	partial	no
ComplexMCP	curated	none	sandboxes	no	no	state-based	no
AGB (this work)	from schemas	9 motifs	yes (STB)	partial	yes	yes	yes

Table 6: The Tool Graph as an object: builder parameters as configured in the released code, the StableToolBench catalog it was built from, and the part of the graph that the validated AGB items use (tools, tool-level dependency edges and, where the sampler kept them, the port-level edges with their κ scores and provenance labels). λ=0 in the reported configuration, so the generic-output penalty was inactive. The complete edge set of the catalog graph is not reported: no committed artifact holds it, only the subgraphs that reached validation.

Quantity	Value
Builder configuration	
$(\alpha, \beta, \gamma, \delta, \lambda)$	(1, 1, 1, 1, 0)
Edge threshold τ (on a sum of four unit-range terms)	1.5
Provenance thresholds (name exact / name similar / description)	0.95 / 0.8 / 0.75
Embedding model	`paraphrase-MiniLM-L3-v2`
Generic output names penalized (λ)	`data`, `result`, `results`, `value`, `message`, `code`
Catalog (StableToolBench)	
Tools	4,070
with input / output / both port sets	3,093 / 4,062 / 3,085
Input ports / output ports (leaf fields)	7,871 / 14,923
Graph used by the 2,180 validated AGB *items*	
Distinct tools	931
Distinct tool-level dependency edges	4,428
Items carrying port-level edges	1,707 of 2,180
Distinct port-level edges	3,455
provenance labels	alias 107, exact 2,402, nested 37, semantic 602, similar 426, similar description 4
κ min / median / max	1.80 / 1.93 / 2.35

$$C_{\text{edge}}(x_q) = \frac{|\{(u,v) \in E_K : (\Phi(u), \Phi(v)) \in E_q\}|}{|\{(u,v) \in E_K : \Phi(u), \Phi(v) \text{ defined}\}|}.$$

The instruction passes if $C_{\text{cov}} \geq \theta_{\text{cov}}$, $C_{\text{red}} \leq \theta_{\text{red}}$, and $C_{\text{edge}} \geq \theta_{\text{edge}}$.

D JUDGE PROMPT

Each LLM judge receives the following prompt template:

> Given the user query, is this tool necessary to complete the task, helpful but optional, or completely unnecessary? Respond with one of: NECESSARY / OPTIONAL / UNNECESSARY and a one-sentence justification.
>
> Query: `{query}`
> Tool name: `{tool_name}`
> Tool description: `{tool_description}`

E SUBGRAPH TOPOLOGIES FORMALIZATION

This appendix gives the formal definition of each of the nine motifs and separates the compatibility edges of the Tool Graph from the executable plan of a task.

Compatibility edges versus executable plans. An edge $(u \to v) \in E$ of the Tool Graph records *compatibility*: some output port of u can feed some input port of v (Section 2). It is not by itself a data dependency that must be realised. A benchmark task is therefore the pair (G_q, π_q): the *task subgraph* $G_q = (V_q, E_q) \subseteq G$ that describes which tools are involved and how they *may* compose, and an *execution plan* $\pi_q = (t^{(1)}, \ldots, t^{(m)})$, a linear order of V_q in which every tool's inputs are bound from the outputs of earlier tools it is compatible with. Equivalently, π_q selects a spanning DAG of G_q (the compatibility edges that point forward in the order) and is one of its topological orders. All validators of Section 3, the reference answer a_q and the execution levels shipped with each item are defined on π_q; the motif class describes the *space of admissible plans* that G_q leaves open. For the seven acyclic motifs π_q is simply a topological order of G_q and the executor derives it by Kahn layering. For the two motifs with directed cycles the plan is fixed at sampling time: for a **Cycle** $v_1 \to \cdots \to v_k \to v_1$, $\pi_q = (v_1, \ldots, v_k)$ is the Hamiltonian path obtained by dropping the closing edge $(v_k \to v_1)$; that edge is retained in G_q because it documents that the last tool's output is compatible with the first tool's input, the round-robin or feedback pattern an agent could exploit, but it is never executed; for a **Clique** π_q is the sampled linear order, which is a topological order of the spanning DAG that keeps, for each pair, the edge consistent with that order. The released executor implements exactly this: when Kahn layering does not cover V_q (a directed cycle is present) it executes the tools one per level in the listed order. In the released data every one of the 323 validated Cycle items and 341 validated Clique items has execution levels equal to its listed order. Reporting motif-level difficulty for Cycle and Clique thus measures how well an agent recovers a tool set whose plan is one of several admissible orders, not whether it can satisfy an unsatisfiable set of dependencies.

We classify task subgraphs into nine canonical *motif classes*. The classification is based on the following structural parameters of G_q:

- acyclicity of G_q;
- acyclicity of underlying skeleton;
- $\mathrm{depth}(G_q)$: number of vertices on the longest directed path in G_q;
- $\mathrm{width}(G_q)$: maximum number of vertices at any single depth level under BFS layering;
- $\mathrm{fan\text{-}out}$: existence of a vertex v with out-degree $d^+(v) \geq 2$;
- $\mathrm{fan\text{-}in}$: existence of a vertex v with in-degree $d^-(v) \geq 2$.

If the skeleton is acyclic, G_q is a *polytree*; otherwise its skeleton contains at least one undirected cycle. Rhombus, Cycle and Clique have cyclic skeleton; all other structures are polytrees. Table 7 summarises the defining constraints. Note that Cycle and Clique are the only classes whose G_q is not a DAG; they are distinguished from the remaining seven acyclic classes by the presence of directed cycles, and their executable plan π_q is a spanning path as described above. Cycle and Clique are allowed to overlap: a clique of size 2 (two tools each compatible with the other's output) is simultaneously a directed cycle of length 2. The two-tool *bridge* items produced by an early sampler are chains with $|V_q| = 2$ and are counted as Chain throughout.

Definition 1 (Chain). *G_q is a* chain *if every vertex has in-degree at most 1 and out-degree at most 1:* $\forall v \in V_q : \quad d^-(v) \leq 1 \ \wedge \ d^+(v) \leq 1.$

A chain requires strictly sequential tool composition. It is the simplest motif and the baseline for linear pipelines.

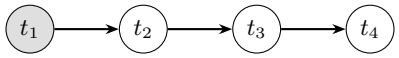


Figure 3: Chain ($|V_q| = 4$, depth $= 4$, width $= 1$).

Definition 2 (Star-out). *G_q is a* star-out *if it is a polytree,* $\mathrm{depth}(G_q) = 2$*, and* $\exists\, r \in V_q$ *with* $d^+(r) \geq 2$ *such that all vertices* $v \neq r$ *satisfy* $d^+(v) = 0$ *(sinks).*

Table 7: Taxonomy of the nine motif classes. "Polytree" means the skeleton (underlying undirected graph) is acyclic; "DAG" means G_q itself has no directed cycle. Depth and width are measured in vertices; a dash (–) denotes an unconstrained parameter. For Cycle and Clique the executable plan π_q is a spanning path of G_q (see text).

Motif	Polytree	DAG	fan-out	fan-in	depth	width
Chain	yes	yes	no	no	≥ 2	1
Star-out	yes	yes	yes	no	2	≥ 2
Star-in	yes	yes	no	yes	2	≥ 2
Rhombus	no	yes	yes	yes	3	2
Hourglass	yes	yes	yes	yes	3	≥ 2
Tree-out	yes	yes	yes	no	≥ 3	≥ 2
Tree-in	yes	yes	no	yes	≥ 3	≥ 2
Cycle	no	no	no	no	–	–
Clique	no	no	yes	yes	–	–

Star-out arises when a single tool produces outputs consumed independently by multiple downstream tools.

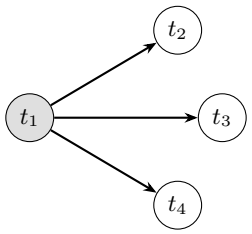


Figure 4: Star-out ($|V_q| = 4$, depth $= 2$, width $= 3$). Shaded node is the root.

Definition 3 (Star-in)**.** *G_q is a* star-in *if it is a polytree,* $\mathrm{depth}(G_q) = 2$*, and* $\exists\, s \in V_q$ *with* $d^-(s) \geq 2$ *such that all vertices* $v \neq s$ *satisfy* $d^-(v) = 0$ *(sources).*

Star-in arises when multiple independent tools gather information aggregated by a single downstream tool.

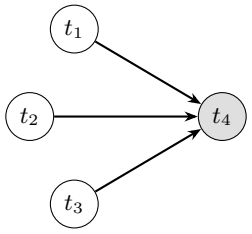


Figure 5: Star-in ($|V_q| = 4$, depth $= 2$, width $= 3$). Shaded node is the sink.

Definition 4 (Rhombus)**.** *G_q is a* rhombus *if its skeleton contains a cycle,* $\mathrm{depth}(G_q) = 3$*, and it contains exactly one fan-out vertex (*$d^+(u) \geq 2$*) and exactly one fan-in vertex (*$d^-(v) \geq 2$*).*

A rhombus is the simplest non-polytree motif. Its skeleton cycle arises because two directed paths share both a source (u) and a sink (v), making the subgraph a DAG but not a polytree.

Definition 5 (Hourglass)**.** *G_q is an* hourglass *if it is a polytree,* $\mathrm{depth}(G_q) = 3$*, and there exists an vertex v such that* $d^-(v) \geq 2$ *and* $d^+(v) \geq 2$*.*

An hourglass combines a convergence layer (fan-in into v) and a divergence layer (fan-out from the same v): one tool aggregates several inputs and its output is consumed by several downstream tools. What distinguishes it from a rhombus is not depth (both have depth 3) but that fan-in and fan-out

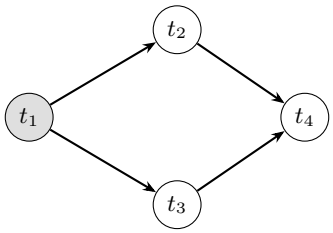


Figure 6: Rhombus ($|V_q| = 4$, depth $= 3$, width $= 2$). Skeleton cycle: $t_1{-}t_2{-}t_4{-}t_3{-}t_1$.

occur at a *single* vertex, so the skeleton stays acyclic (a polytree), whereas a rhombus splits at one vertex and re-joins at another and therefore has a cyclic skeleton.

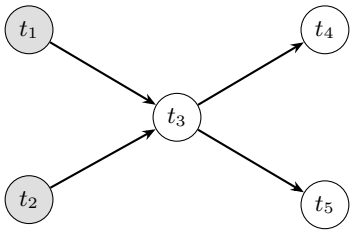


Figure 7: Hourglass ($|V_q| = 5$, depth $= 3$, width $= 2$). Vertex t_3: $d^-(t_3) = 2$, $d^+(t_3) = 2$.

Trees are polytrees with non-trivial depth and width that contain either only fan-out vertices or only fan-in vertices among their high-degree nodes. This mirrors the Star-out / Star-in distinction, but at depth ≥ 3.

Definition 6 (Tree-out)**.** *G_q is a* tree-out *if*

1. *its skeleton is acyclic (polytree);*
2. $\mathrm{depth}(G_q) \geq 3$*;*
3. $\mathrm{width}(G_q) \geq 2$*;*
4. *every vertex of degree ≥ 3 is a* fork*: $d^+(v) \geq 2$.*

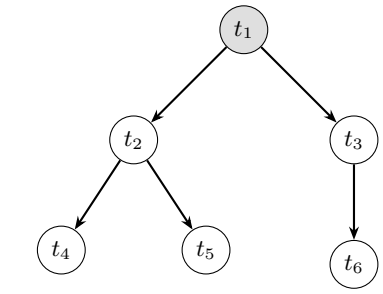
Definition 7 (Tree-in)**.** *G_q is a* tree-in *if*

1. *its skeleton is acyclic (polytree);*
2. $\mathrm{depth}(G_q) \geq 3$*;*
3. $\mathrm{width}(G_q) \geq 2$*;*
4. *every vertex of degree ≥ 3 is a* join*: $d^-(v) \geq 2$.*

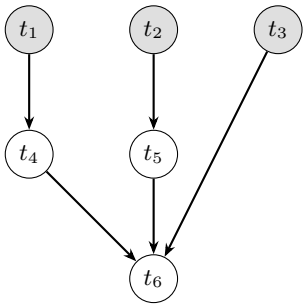
Characterisation via BFS. Tree-out and Tree-in admit a natural generative characterisation. A *tree-out* is precisely the class of subgraphs obtainable by a BFS traversal starting from a source vertex and following edges in their directed orientation. Dually, a *tree-in* is precisely the class of subgraphs obtainable by a BFS traversal starting from a sink vertex and following edges *against* their directed orientation. This characterisation motivates our subgraph sampler (Section 3), which generates tree motifs by BFS from sources (tree-out) or sinks (tree-in).

Definition 8 (Cycle)**.** *G_q is a* cycle *if its edge set forms a single simple directed cycle: there exists an ordering $(v_1, v_2, \ldots, v_k)$ of V_q such that $E_q = \{(v_1{\to}v_2), (v_2{\to}v_3), \ldots, (v_{k-1}{\to}v_k), (v_k{\to}v_1)\}$ with $k = |V_q| \geq 2$.*

A cycle represents a set of tools in sequential compatibility: each tool can consume the output of its predecessor and produce input for its successor, and the last tool's output is compatible with the



(a) Tree-out



(b) Tree-in

Figure 8: Trees ($|V_q| = 6$, depth $= 3$, width $= 3$). (a) Tree-out: shaded root t_1 fans out; all high-degree vertices are forks. (b) Tree-in: shaded sink t_6 fans in; all high-degree vertices are joins. Neither skeleton contains a cycle.

first tool's input. This motif arises naturally when tools form a feedback or round-robin pipeline. Its executable plan is the Hamiltonian path $\pi_q = (v_1, \ldots, v_k)$: the closing edge $(v_k \to v_1)$ is part of G_q but is not executed, so the reference answer is the output of v_k. Note that a cycle of length 2 ($t_1 \to t_2 \to t_1$) is also a minimal clique; cycles of length ≥ 3 are distinct from all cliques.

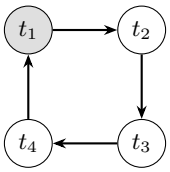


Figure 9: Cycle ($|V_q| = 4$, length $= 4$). Executed plan: t_1, t_2, t_3, t_4; the closing edge $t_4 \to t_1$ is compatibility only.

Definition 9 (Clique)**.** *G_q is a* clique *if its skeleton is the complete graph $K_{|V_q|}$: for every pair of distinct vertices $\{t_i, t_j\}$ at least one of $(t_i \to t_j)$, $(t_j \to t_i)$ is in E_q.*

A clique arises when every pair of tools in a domain is mutually compatible, for example when a shared `keyword` or `symbol` field flows freely between search, monitoring and analytics tools. Cliques are sampled on the undirected view of the Tool Graph and then carry all directed compatibility edges between the chosen tools; in the released AGB items 199 of the 341 validated cliques (58%) have both directions for every pair, the rest have at least one. Whenever $|V_q| \geq 3$ the subgraph contains directed cycles, so its executable plan π_q is the sampled linear order (Definition above), which is a topological order of the spanning DAG formed by the forward edges. Since any order is admissible, a clique is the motif with the largest space of admissible plans; the agent is scored on recovering the tool set, not a particular order.

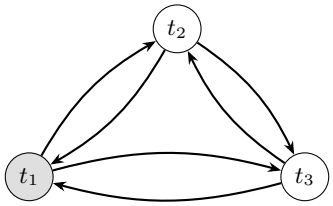


Figure 10: Clique ($|V_q| = 3$) in which every pair is compatible in both directions; all $3{\times}2 = 6$ directed edges are present. A clique only requires at least one direction per pair.

For $|V_q| \geq 3$, a clique strictly contains multiple directed cycles but is not itself a simple cycle; a simple cycle of length ≥ 3 has a skeleton with $|V_q|$ edges rather than $\binom{|V_q|}{2}$ and is therefore not a clique.

F AUDIT NECESSITY RESULTS

Table 8 gives, for each LLM judge and for the strict-majority human label, the distribution of the three necessity labels over the human-study (query, tool) pairs; the judge labels and annotator files are released with the repository and the table is rendered from them.

Table 8: Three-way necessity distribution (% NEC./% OPT./% UNNEC.) of StableToolBench reference tools on the human-study (query, tool) pairs, per LLM judge and for the strict-majority human label (pairs with a 2-of-3 majority). *Non-ess.* = OPT. + UNNEC. Cells are percentages ± the half-width of the 95% percentile-bootstrap CI over pairs (B=10,000).

Labeler	n	**% Nec.**	**% Opt.**	**% Unnec.**	**% Non-ess.**
Claude Haiku 4.5	200	55.0 ±7.0	18.5 ±5.2	26.5 ±6.0	45.0 ±7.0
GPT-5-Mini	200	41.5 ±6.8	31.0 ±6.5	27.5 ±6.3	58.5 ±6.8
Qwen3-235B-A22B	200	50.0 ±7.0	15.0 ±5.0	35.0 ±6.5	50.0 ±7.0
Human (strict majority)	189	65.6 ±6.9	8.5 ±4.0	25.9 ±6.1	34.4 ±6.9

G GENERATION ACCEPTANCE RESULTS

Table 9 gives the per-motif sampling, generation and validation counts behind AGB.

Table 9: Per-motif generation acceptance and structural complexity. *Sampled* is the number of unique subgraphs drawn by the motif samplers (Section 2); *Generated* is the subset for which an instruction was produced by the plan-first pipeline (Section 3) – every sampled subgraph received an instruction, so the two coincide; *Validated* is the subset that passes V_{exec} on the StableToolBench runtime and constitutes AGB (2,180 of 3,032); *Strict* is the subset with no answer-removable reference tool under the answer-grounded ablation (Table 11), i.e. AGB-STRICT. The two-tool *bridge* items are counted as Chain with $|G_q|$=2. V_{sem} and $V^*_{\min}$ are LLM-judged soft validators carried with the generated items: V_{sem} requires instruction-vs-subgraph KG coverage to be adequate; $V^*_{\min}$ is the fraction of items whose tool list contains no tool the judge marks unnecessary – a judge-based proxy, distinct from the structural-redundancy and answer-grounded minimality tests. V_{exec} is the executable rate (= Validated / Generated); daggered (†) values are computed over a partial run. $\overline{\text{depth}}$ and $\overline{\text{width}}$ are the mean longest-path depth (in vertices) and mean BFS-layer width of the sampled subgraphs.

Motif	$\lvert G_q\rvert$	**Sampled**	**Generated**	**Validated**	**Strict**	V_{sem}	$V^*_{\min}$	V_{exec}	$\overline{\text{depth}}$	$\overline{\text{width}}$
Chain	2–10	547	547	431	59	0.96	0.33	0.79	5.29	1.00
Star (out)	3–8	300	300	229	97	0.98	1.00	0.76	2.00	4.50
Star (in)	3–8	300	300	249	24	0.97	1.00	0.83	2.00	4.50
Rhombus	4	300	300	142	22	0.96	1.00	0.47	3.00	2.00
Hourglass	5	100	100	74	48	0.46	0.05	0.74	3.30	2.30
Tree (out, BFS)	5–10	300	300	179	68	0.91	0.23	0.60	4.00	2.98
Tree (in, BFS)	5–10	300	300	212	38	0.93	0.21	0.71	4.00	5.87
Clique	2–10	450	450	341	49	0.78	0.49	0.76	6.00	5.00
Cycle	2–10	435	435	323	46	0.46	0.37	0.74	5.86	1.89

H STRUCTURAL REDUNDANCY BY MOTIF

Table 10 reports the pure-topology redundancy test of Section 3 ($V^{\text{struct}}_{\min}$) on the validated AGB items. The table is rendered by `scripts/render/agg_exec_minimality.py` from `data/processed/structural_minimality.jsonl`; it is not an answer-grounded minimality test.

Table 11 reports the complementary *answer-grounded* test on the same items: a reference tool is answer-removable if re-executing the plan without it (original external arguments replayed, sinks never removed) still yields a sink answer with the recorded keys

Table 10: **Structural redundancy** of AGB items (validated; *bridge* counted as Chain) by motif ($V_{\min}^{\text{struct}}$; not answer-grounded minimality). A tool is *structurally redundant* iff it is not a sink and every input port it feeds is also produced by another tool in the subgraph, i.e. an alternative producer exists for every port. *% Reducible* is the fraction of items with at least one such tool; $\overline{|\text{min}|}$ is the mean size after greedily removing redundant tools; *Reduction* is $\overline{|GT|} - \overline{|\text{min}|}$. The criterion is *1 by construction* for Star-in (all sources feed one sink port) and Clique (every port has several producers), so high values there describe the motif, not annotation noise. Whether a redundant tool is also *answer*-irrelevant is a separate, execution-grounded test.

Source	Slice	n	$\overline{\lvert GT\rvert}$	% Reducible	$\overline{\lvert\text{min}\rvert}$	Reduction
StableToolBench	all	—	—	—	—	—
AGB-Generated	Chain	431	5.3	0.0	5.3	0.0
	Star (out)	229	5.3	0.0	5.3	0.0
	Star (in)	249	5.4	96.4	2.3	3.1
	Rhombus	142	4.0	73.9	3.3	0.7
	Hourglass	74	5.0	40.5	4.5	0.5
	Tree (out, BFS)	179	8.7	0.0	8.7	0.0
	Tree (in, BFS)	212	9.3	100.0	5.2	4.1
	Clique	341	5.5	84.8	2.1	3.4
	Cycle	323	5.4	0.0	5.4	0.0
AGB-Generated	all	2180	5.9	40.2	4.6	1.3

and value types. Items with no answer-removable tool form AGB-STRICT (451 of 2,180); the table is rendered by `scripts/render/agg_exec_minimality.py` from `data/processed/answer_minimality.jsonl`.

Table 11: **Answer-grounded minimality** of AGB items (validated; *bridge* counted as Chain) by motif, structural-answer criterion: a tool is *answer-removable* iff re-executing the plan without it (original external arguments replayed, sinks never removed) succeeds and yields a sink answer with the same keys and value types as the recorded one. *% Reducible* is the fraction of items with at least one such tool; $\overline{|\text{min}|}$ is computed under the exact criterion (greedy). Items with no answer-removable tool form AGB-STRICT.

Source	Slice	n	$\overline{\lvert GT\rvert}$	% Reducible	$\overline{\lvert\text{min}\rvert}$	Reduction
StableToolBench	all	—	—	—	—	—
AGB-Generated	Chain	431	5.3	86.3	2.4	2.9
	Star (out)	229	5.3	57.6	5.1	0.3
	Star (in)	249	5.4	90.4	2.6	2.8
	Rhombus	142	4.0	84.5	2.2	1.8
	Hourglass	74	5.0	35.1	4.1	0.9
	Tree (out, BFS)	179	8.7	62.0	7.8	0.9
	Tree (in, BFS)	212	9.3	82.1	4.0	5.4
	Clique	341	5.5	85.6	2.5	3.0
	Cycle	323	5.4	85.8	2.3	3.1
AGB-Generated	all	2180	5.9	79.3	3.3	2.6

I CLEANING STABLETOOLBENCH RESULTS

Table 12 gives the per-validator pass rates behind the STB-CLEAN slice of Section 5.2.

Table 12: Cleaning StableToolBench: per-validator marginal pass rates on the original 765-query benchmark, joint pass rate (STB-Clean), and resulting size. V_{exec} is derived from the cleared-API subset (a record passes if its tools are reachable in /virtual). V_{sem} and V^*_{min} are LLM-judged (`validation.is_adequate` and zero unnecessary GT tools respectively). Other validators (V_{sch}, V_{ans}, V_{sc}, V_{stab}) are not yet computed on STB.

Source	V_{sch}	V_{exec}	V_{ans}	V^*_{min}	V_{sc}	V_{sem}	V_{stab}	**All**	|Orig|	|Clean|	**Ret. %**
StableToolBench	—	0.732	—	0.268	—	0.818	—	0.222	765	170	22.2

J BOOTSTRAP CONFIDENCE INTERVALS

All point estimates reported in the main text are accompanied by 95% percentile-bootstrap confidence intervals (B=10,000 resamples, seed fixed). Items being resampled are benchmark records for the recall/success-rate tables and (query, tool) pairs for the judge-agreement table. Paired comparisons (within-record Δ = Adj − Raw; within-pair $\Delta\kappa$ between judges) resample indices jointly. Two-sided p-values are $2 \cdot \min(\Pr(\Delta^* \leq 0), \Pr(\Delta^* \geq 0))$, clamped at $2/B = 2\times10^{-4}$. Full machine-readable CIs and bootstrap draws are available in `results/bootstrap_ci.json`.

$\Delta\kappa$ between judges vs. human (paired). On the 189 strict-majority human-study pairs, the Claude-vs-GPT-5-Mini gap is $\Delta\kappa = +.111\,[+.004, +.214]$; the CI just clears zero. The other two paired contrasts straddle zero: Claude vs. Qwen3-235B-A22B $+.071\,[-.041, +.178]$ and GPT vs. Qwen $-.041\,[-.145, +.065]$. We read this as "no judge is reliably the ground-truth oracle," which justifies treating the three judges as exchangeable when forming consensus.

Δ between AGB and noisy benchmarks (unpaired, per agent). For every agent, the AGB raw-recall CI in Table 13 sits strictly above the STB and STB-EXEC raw-recall CIs (the smallest gap, Claude on STB-EXEC, is .861 vs. .588). The comparison is unpaired and the substrates differ, so we use it only to bracket the magnitude of the GT-noise effect, not as a direct ranking signal.

Per-cell CIs: cross-agent recall. The STB-CLEAN rows are the 170 all-validator queries inside the StableToolBench run (no separate run). On that slice, whose V^*_{min} filter removes non-minimal reference sets, raw recall is .81 for every agent and Δ shrinks to $+.04$–$+.05$, as the filter is designed to make it.

Table 13: Per (agent, benchmark) cell: raw and adjusted recall and paired Δ = Adj − Raw, each with a 95% bootstrap CI. AGB cells are degenerate by construction; every Δ-CI on STB and STB-EXEC lies strictly above zero.

Agent	**Benchmark**	**Raw**	**Adj.**	Δ = Adj − Raw	p
Claude Haiku 4.5	StableToolBench	.532 [.511, .552]	.789 [.769, .808]	+.258 [.238, .277]***	$<.001$
	STB-Clean	.811 [.767, .852]	.853 [.811, .892]	+.044 [.022, .068]***	$<.001$
	STB-Exec	.563 [.538, .588]	.811 [.789, .832]	+.249 [.226, .272]***	$<.001$
	AGB-Generated	.870 [.859, .880]	.870 [.859, .880]	$\equiv 0$	—
Gemma-4-31B-IT	StableToolBench	.524 [.503, .545]	.786 [.767, .805]	+.263 [.243, .283]***	$<.001$
	STB-Clean	.814 [.771, .856]	.860 [.820, .899]	+.048 [.025, .073]***	$<.001$
	STB-Exec	.543 [.518, .568]	.792 [.769, .815]	+.251 [.227, .274]***	$<.001$
	AGB-Generated	.910 [.902, .919]	.910 [.902, .919]	$\equiv 0$	—
Qwen3.6-Flash	StableToolBench	.529 [.508, .549]	.792 [.773, .810]	+.265 [.246, .285]***	$<.001$
	STB-Clean	.813 [.770, .855]	.862 [.822, .900]	+.050 [.027, .075]***	$<.001$
	STB-Exec	.547 [.522, .572]	.807 [.785, .828]	+.260 [.238, .284]***	$<.001$
	AGB-Generated	.929 [.924, .935]	.929 [.924, .935]	$\equiv 0$	—
Ministral-14B	StableToolBench	.535 [.515, .556]	.790 [.771, .809]	+.256 [.237, .276]***	$<.001$
	STB-Clean	.813 [.770, .854]	.854 [.813, .893]	+.042 [.021, .067]***	$<.001$
	STB-Exec	.556 [.532, .580]	.800 [.779, .821]	+.245 [.223, .268]***	$<.001$
	AGB-Generated	.886 [.877, .894]	.886 [.877, .894]	$\equiv 0$	—

Table 14: Per (agent, motif) success rate on AGB with a 95% bootstrap CI. The macro-average CI is a bootstrap over all items, not the mean of per-motif point estimates.

Agent	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	Clique	Cycle	Macro avg.
Claude Haiku 4.5	.411 [.364, .457]	.811 [.759, .859]	.834 [.786, .882]	.866 [.810, .923]	.622 [.514, .730]	.514 [.441, .587]	.373 [.307, .439]	.704 [.657, .751]	.697 [.644, .746]	.631 [.611, .651]
Gemma-4-31B-IT	.456 [.409, .504]	.877 [.833, .917]	.930 [.895, .961]	.888 [.828, .940]	.554 [.446, .662]	.614 [.538, .690]	.493 [.425, .560]	.782 [.737, .826]	.786 [.740, .830]	.701 [.681, .721]
Qwen3.6-Flash	.472 [.426, .519]	.928 [.895, .960]	.956 [.930, .983]	.976 [.944, 1.000]	.608 [.500, .716]	.593 [.520, .667]	.505 [.439, .571]	.666 [.616, .716]	.721 [.672, .771]	.691 [.672, .710]
Ministral-14B	.450 [.404, .497]	.771 [.719, .823]	.795 [.742, .843]	.873 [.817, .923]	.500 [.392, .608]	.455 [.382, .528]	.387 [.321, .453]	.745 [.698, .789]	.687 [.638, .737]	.628 [.608, .648]

Per-cell CIs: cross-model motif success.

Table 15: Per (agent, $|G_q|$) success rate on AGB with a 95% bootstrap CI.

Agent	$\|G_q\|$=2	$\|G_q\|$=3	$\|G_q\|$=4	$\|G_q\|$=5	$\|G_q\|$=6	$\|G_q\|$=7	$\|G_q\|$=8
Claude Haiku 4.5	.950 [.921, .975]	.909 [.868, .945]	.788 [.743, .830]	.690 [.634, .742]	.622 [.555, .684]	.526 [.460, .592]	.473 [.407, .540]
Gemma-4-31B-IT	.964 [.938, .990]	.952 [.924, .981]	.841 [.802, .879]	.732 [.679, .780]	.726 [.663, .788]	.623 [.557, .689]	.600 [.533, .662]
Qwen3.6-Flash	.960 [.930, .985]	.941 [.909, .973]	.859 [.821, .894]	.756 [.707, .805]	.713 [.651, .775]	.596 [.531, .662]	.564 [.498, .631]
Ministral-14B	.926 [.886, .960]	.900 [.859, .941]	.782 [.740, .824]	.648 [.592, .704]	.656 [.589, .718]	.512 [.441, .577]	.502 [.436, .569]

Per-cell CIs: cross-model size success.

Table 16: LLM-vs-LLM κ on the 200-pair human-study set, and per-judge κ / macro-F1 against the strict-majority human label on n=189 pairs, each with a 95% bootstrap CI. Inter-human row bootstraps over the 12 annotator-pair κ values.

Judge	κ vs. other LLMs (per pair)	κ vs. human	Macro-F1 vs. human
Claude Haiku 4.5	GPT-5-Mini: .556 [.458, .647]; Qwen3-235B-A22B: .529 [.428, .623]	.648 [.549, .743]	.703 [.628, .775]
GPT-5-Mini	Claude: .556 [.458, .647]; Qwen3-235B-A22B: .531 [.433, .622]	.537 [.444, .631]	.654 [.588, .717]
Qwen3-235B-A22B	Claude: .529 [.428, .623]; GPT-5-Mini: .531 [.433, .622]	.578 [.479, .674]	.640 [.568, .710]
Inter-human (avg. pairwise)	—	.439 [.351, .530]	—

Per-cell CIs: judge agreement.

K JUDGE RELIABILITY AND HUMAN-ONLY LABELS

This appendix carries the reliability evidence behind the adjusted-recall gap of Table 2: the gap recomputed with human-only labels, judge–human agreement under a two-class collapse, the alternative annotator test, and the effect of showing the judge the full reference set.

Judge agreement (Table 17). Judge–judge κ is .55–.56 on the 189 strict-majority pairs with overlapping CIs; against the human consensus the Claude-vs-GPT-5-Mini paired gap $\Delta\kappa = +.111\,[+.004, +.214]$ just clears zero and the other two contrasts do not. Disagreement concentrates on the OPTIONAL class, which motivates the two-class collapse below.

Human-only labels (Table 18). On the 189 human-study pairs with a strict-majority human label, restricting the reference set to the tools the annotators mark NECESSARY raises recall by $+.24$ to $+.26$ for every agent (95% CIs exclude $+.18$); the three-judge definition gives $+.25$ to $+.29$ on the same pairs. The gap is therefore not an artefact of the LLM-judge definition. The four agents' adjusted recalls lie within $.04$ of each other, so no agent ordering is claimed from this metric.

Two-class agreement (Table 19). Adjusted recall only needs NECESSARY vs. not, so we also report κ and macro-F1 after collapsing OPTIONAL and UNNECESSARY; agreement rises for every judge under the collapse and on the unanimous pairs.

Alternative annotator test (Table 20). Under the alt-test of Calderon et al. (2025), Claude Haiku 4.5 passes at ε=0.10 (exactly at ω=0.50) and 0.15, Qwen3-235B at ε=0.15, and GPT-5-Mini only at ε=0.20; advantage probabilities are 0.81–0.91 for all three, i.e. each judge represents the remaining annotators at least as well as a left-out annotator on over 80% of pairs. Allowing the judge to abstain

Table 17: Inter-judge agreement on three-way tool-necessity classification. The middle column reports each judge's Cohen's κ against each of the other two LLM judges (computed over the n=189 human-study pairs with a strict-majority human label; $\pm$ is the half-width of the 95% percentile-bootstrap CI, B=10,000). The right two columns report Cohen's κ and macro-F1 of each judge against the strict-majority label of three human votes per pair (six annotators, round-robin assignment) on the same 200 pairs; 11 pairs are dropped for lacking a 2-of-3 majority, so the kept set is n=189. The last row gives the average pairwise inter-human κ across overlapping (annotator, annotator) slices, with a CI bootstrapped over the 12 annotator-pair κ values.

Judge	**κ vs. other LLMs**	**κ vs. human**	**Macro-F1 vs. human**
Claude Haiku 4.5	.555 $\pm$.098 / .562 $\pm$.101	.648 $\pm$.098	.703 $\pm$.071
GPT-5-Mini	.555 $\pm$.097 / .554 $\pm$.097	.537 $\pm$.093	.654 $\pm$.064
Qwen3-235B-A22B	.562 $\pm$.100 / .554 $\pm$.097	.578 $\pm$.099	.640 $\pm$.074
Inter-human (avg. pairwise)	—	.439 $\pm$.089	—

Table 18: Does the raw→adjusted recall gap depend on who defines NECESSARY? Pair-level analysis on the human-study (query, tool) pairs of StableToolBench (stb-original): *raw* is the share of labelled reference tools the agent selected; *adjusted* is the same share over the tools a label source marks NECESSARY; Δ = adjusted − raw with a 95% bootstrap CI (pairs resampled jointly, B=10,000). n / n_{nec} = labelled pairs / pairs marked necessary.

Agent	**Label source**	n / n_{nec}	**Raw**	**Adjusted**	Δ
Claude Haiku 4.5	human strict-majority	189 / 124	.519	.758	+.240 $\pm$.058
	human unanimous	118 / 91	.669	.868	+.199 $\pm$.068
	judge strict-majority	193 / 101	.513	.762	+.249 $\pm$.068
	Table 3 consensus	200 / 119	.505	.756	+.251 $\pm$.061
Gemma-4-31B-IT	human strict-majority	189 / 124	.508	.750	+.242 $\pm$.057
	human unanimous	118 / 91	.661	.857	+.196 $\pm$.067
	judge strict-majority	193 / 101	.492	.772	+.280 $\pm$.069
	Table 3 consensus	200 / 119	.485	.756	+.271 $\pm$.061
Qwen3.6-Flash	human strict-majority	188 / 123	.527	.789	+.262 $\pm$.059
	human unanimous	117 / 90	.701	.911	+.210 $\pm$.072
	judge strict-majority	192 / 100	.510	.800	+.290 $\pm$.068
	Table 3 consensus	199 / 118	.503	.780	+.277 $\pm$.061
Ministral-14B	human strict-majority	189 / 124	.524	.774	+.250 $\pm$.059
	human unanimous	118 / 91	.669	.868	+.199 $\pm$.067
	judge strict-majority	193 / 101	.513	.772	+.259 $\pm$.069
	Table 3 consensus	200 / 119	.505	.782	+.277 $\pm$.062

Table 19: Judge–human agreement under the three-class scheme and the two-class collapse (NECESSARY vs. {OPTIONAL, UNNECESSARY}), on pairs with a strict human majority and on unanimous pairs. Cells: statistic $\pm$ 95% bootstrap half-width over pairs (B=10,000).

Subset	**Judge**	κ (3-class)	F1 (3-class)	κ (2-class)	F1 (2-class)
strict majority (n=189)	Claude Haiku 4.5	.648 $\pm$.098	.703 $\pm$.074	.646 $\pm$.110	.821 $\pm$.056
	GPT-5-Mini	.537 $\pm$.093	.654 $\pm$.064	.521 $\pm$.107	.749 $\pm$.061
	Qwen3-235B-A22B	.578 $\pm$.097	.640 $\pm$.072	.605 $\pm$.108	.798 $\pm$.058
unanimous (n=118)	Claude Haiku 4.5	.748 $\pm$.124	.740 $\pm$.110	.751 $\pm$.132	.874 $\pm$.069
	GPT-5-Mini	.535 $\pm$.126	.647 $\pm$.069	.495 $\pm$.139	.731 $\pm$.085
	Qwen3-235B-A22B	.717 $\pm$.128	.700 $\pm$.090	.716 $\pm$.135	.855 $\pm$.070

removes 1–13% of pairs without raising agreement. We therefore rely on the three-judge consensus anchored by the human study, not on any single judge as a replacement for human annotation.

Table 20: Alternative Annotator Test (alt-test; Calderon et al., 2025) of each LLM judge against the six human annotators on the 200 human-study pairs. ρ is the average advantage probability (how often the judge agrees with the other annotators at least as well as the left-out annotator does); ω is the winning rate, the share of annotators for whom the judge is significantly at least as aligned (one-sided paired t-test with cost–benefit margin ε, Benjamini–Yekutieli at q=0.05); the judge passes iff $\omega \geq 0.5$. *Abstain* rows use the prompt variant that allows the judge to abstain; abstained pairs are excluded from that judge's test.

Prompt	Judge	ρ	ω (ε=0.10)	ω (ε=0.15)	ω (ε=0.20)	abstain
standard	Claude Haiku 4.5	.908	.50✓	1.00✓	1.00✓	—
	GPT-5-Mini	.813	.00	.33	.67✓	—
	Qwen3-235B-A22B	.885	.17	.83✓	1.00✓	—
abstain	Claude Haiku 4.5	.902	.50✓	1.00✓	1.00✓	.01
	GPT-5-Mini	.786	.00	.00	.50✓	.01
	Qwen3-235B-A22B	.851	.17	.33	.67✓	.13

Plan context (Table 21). Giving the judge the query's full reference set as context lowers agreement with the human majority ($\Delta\kappa = -.12$, 95% CI $[-.22, -.01]$) and shifts 58 of 189 labels, mostly toward UNNECESSARY. Because the annotators judged pairs without plan context, the context-free prompt (Appendix D) is kept; plan-aware necessity is left as future work. For the non-minimality claim this is conservative: a stricter, plan-aware judge would mark more reference tools non-essential.

Table 21: Effect of plan context on judge–human agreement. *Context-free*: the judge sees query, tool name and description (the paper's prompt); *context*: it also sees the query's full reference tool set with the judged tool marked. Same pairs with a strict human majority; cells are statistic $\pm$ 95% bootstrap half-width over pairs, Δ is the paired bootstrap of context $-$ context-free (B=10,000).

Judge	Statistic	Context-free	Context	Δ	p
Claude Haiku 4.5 (n=189)	κ (3-class)	.648 $\pm$.098	.532 $\pm$.095	-.116 $\pm$.102	.027
	macro-F1 (3-class)	.703 $\pm$.071	.640 $\pm$.072	-.064 $\pm$.082	.117
	κ (2-class)	.646 $\pm$.109	.515 $\pm$.107	-.130 $\pm$.117	.032
	macro-F1 (2-class)	.821 $\pm$.057	.744 $\pm$.061	-.077 $\pm$.065	.018

L METRIC ROBUSTNESS

Table 22 reports recall, precision, F1, exact-set match, complete-set success and hallucination rate side by side for every agent and benchmark variant. Precision is .98–.99 in every cell and no agent ever selects a tool outside the 25-tool candidate list, so the complete-set success of Tables 3–4 cannot be attributed to over-selection; agents select fewer tools than the reference in every cell. Exact-set match differs from success by at most .05 and ranks motifs identically (Spearman $\rho \geq .98$ per agent; Tables 23–24). F1, which gives partial credit, compresses the motif spread from $\approx .5$ to $\approx .1$ and orders motifs differently (ρ vs. success .33–.85): the motif effect is a property of *complete-set* recovery. The agent ranking also depends on the metric (Gemma-4-31B leads on complete-set metrics, Qwen3.6-Flash on graded ones), so the paper claims no single agent ordering.

Omissions track the instruction, not the runtime. Two notions of necessity label the same reference tools: the $V^*_{\min}$ judge, reading only the instruction, names tools it does not motivate (591 tools in 444 items; none in the stars and Rhombus, 41–42% of Hourglass and Tree-in items), and the answer ablation of Section 5.2 marks tools whose removal leaves the sink answer unchanged. Table 25 shows that every agent omits judge-flagged tools 61–64% of the time against 6–13% for the rest, while answer-removable tools are omitted slightly *less* often than answer-necessary ones ($-.02$ to $-.03$, CIs below zero). Agents drop what the instruction fails to motivate, whether or not the runtime needs it: omission is a text-coverage failure, not an economy, which is also why $V^*_{\min}$ is lowest on Hourglass, where the generator must motivate every joining tool at once. Table 26 adds the judge-flag count to the regression of Table 1: it is the strongest single covariate (-1.25 log-odds per flagged tool; McFadden R^2 .272 $\rightarrow$ .319), motif stays significant (LRT $p \approx 10^{-96}$), Chain keeps

Table 22: Full tool-selection metric suite per agent and benchmark variant (means over items; AGB restricted to validated items, errored runs excluded). *Recall*, *Precision*, *F1* compare the selected set S with the reference set R; *Exact* is $S{=}R$; *Success* is $R \subseteq S$ (the paper's original metric); *Halluc.* is the share of selected tools outside the 25-tool candidate list; $|S|$, $|R|$ are mean set sizes.

Agent	Benchmark	n	Recall	Precision	F1	Exact	Success	Halluc.	$\lvert S\rvert$	$\lvert R\rvert$
Claude Haiku 4.5	StableToolBench	765	0.532	0.990	0.645	0.205	0.210	0.000	2.21	5.32
	STB-Exec	560	0.563	0.992	0.672	0.245	0.246	0.000	2.21	5.13
	AGB-Generated	2180	0.870	0.992	0.896	0.622	0.631	0.000	5.07	5.93
Gemma-4-31B-IT	StableToolBench	765	0.524	0.992	0.638	0.207	0.209	0.000	2.15	5.32
	STB-Exec	560	0.543	0.987	0.653	0.234	0.234	0.000	2.12	5.13
	AGB-Generated	2128	0.910	0.994	0.932	0.694	0.701	0.000	5.35	5.95
Qwen3.6-Flash	StableToolBench	764	0.529	0.993	0.642	0.211	0.213	0.000	2.15	5.31
	STB-Exec	545	0.547	0.994	0.659	0.228	0.228	0.000	2.15	5.17
	AGB-Generated	2159	0.929	0.994	0.956	0.687	0.691	0.000	5.45	5.94
Ministral-14B	StableToolBench	765	0.535	0.983	0.648	0.204	0.212	0.000	2.24	5.32
	STB-Exec	560	0.556	0.988	0.666	0.229	0.232	0.000	2.22	5.13
	AGB-Generated	2179	0.886	0.977	0.909	0.579	0.628	0.000	5.26	5.93

Table 23: Per-motif exact-set-match rate (selected set equals the reference set) on AGB (validated items; the two-tool *bridge* items are Chain with $|G_q|{=}2$). The last column is the macro-average over motifs. Cells are means over records with the half-width of the 95% percentile-bootstrap CI ($B{=}10{,}000$, records resampled within the cell; the macro column resamples within each motif). The *Average* row pools the four agents' items.

Agent	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	Clique	Cycle	Macro avg.
Claude Haiku 4.5	.404 ±.044	.811 ±.048	.834 ±.050	.859 ±.060	.608 ±.108	.514 ±.073	.368 ±.064	.680 ±.050	.684 ±.051	.640 ±.021
Gemma-4-31B-IT	.442 ±.047	.877 ±.044	.926 ±.033	.888 ±.056	.554 ±.108	.614 ±.076	.488 ±.068	.773 ±.044	.771 ±.045	.704 ±.021
Qwen3.6-Flash	.467 ±.048	.928 ±.034	.956 ±.028	.976 ±.028	.608 ±.108	.593 ±.073	.505 ±.066	.657 ±.050	.712 ±.050	.711 ±.020
Ministral-14B	.420 ±.046	.715 ±.056	.755 ±.055	.803 ±.067	.486 ±.108	.421 ±.073	.368 ±.066	.672 ±.050	.610 ±.053	.583 ±.023
Average	.433 ±.023	.832 ±.023	.868 ±.021	.878 ±.028	.564 ±.057	.535 ±.037	.432 ±.033	.695 ±.025	.694 ±.026	.660 ±.011

Table 24: Exact-set-match rate (selected set equals the reference set) on AGB (validated items) as a function of subgraph size $|G_q|$. Cells are means over records with the half-width of the 95% percentile-bootstrap CI ($B{=}10{,}000$, records resampled within the cell). The *Average* row pools the four agents' items.

Agent	$\lvert G_q\rvert{=}2$	$\lvert G_q\rvert{=}3$	$\lvert G_q\rvert{=}4$	$\lvert G_q\rvert{=}5$	$\lvert G_q\rvert{=}6$	$\lvert G_q\rvert{=}7$	$\lvert G_q\rvert{=}8$
Claude Haiku 4.5	.950 ±.030	.891 ±.041	.779 ±.043	.679 ±.054	.617 ±.067	.507 ±.068	.465 ±.064
Gemma-4-31B-IT	.954 ±.028	.948 ±.031	.838 ±.040	.725 ±.051	.721 ±.060	.613 ±.066	.587 ±.064
Qwen3.6-Flash	.955 ±.027	.927 ±.034	.859 ±.037	.749 ±.051	.708 ±.062	.592 ±.066	.564 ±.067
Ministral-14B	.886 ±.042	.841 ±.048	.718 ±.046	.596 ±.056	.608 ±.067	.469 ±.066	.431 ±.067
Average	.936 ±.017	.901 ±.019	.797 ±.021	.687 ±.027	.663 ±.032	.545 ±.033	.512 ±.033

most of its penalty ($-1.48 \rightarrow -1.12$), the Hourglass penalty halves ($-1.20 \rightarrow -0.55$) and Tree-in's residual vanishes; scoring only the judge-necessary tools gives the same picture, and AGB-STRICT agrees. We therefore describe Chain as the robust sequential penalty and about half of the Hourglass penalty as instruction coverage. This analysis was run after the panel, and its rule was fixed after an exploratory look at the data; the report in the repository says so.

M SIZE AND SURFACE CONTROLS, AND THE AGB-STRICT SUBSET

Table 27 restricts the motif comparison to the size window $|G_q| \in [5, 8]$ shared by most motifs; Chain (.21) remains far below the others (.57–.84), while Tree-in (.65) sits with Tree-out (.64) and Clique (.69). Table 28 compares the hardest and easiest raw motifs within question-length quartiles. Tables 29–30 repeat the main motif and size tables on the 451-item AGB-STRICT subset

Table 25: Do agents omit the tools that are not needed? Omission rate of a reference tool by necessity class, validated AGB items. *Judge-flagged*: the $V^*_{\min}$ judge, reading the instruction, named the tool unnecessary; *answer-removable*: re-executing without the tool leaves execution and the sink answer unchanged (sinks excluded from that contrast; their omission rate is given for reference). Differences carry 95% percentile-bootstrap CIs over items (B=10,000).

	Judge flag (instruction)			Answer ablation (runtime)			
Agent	flagged	necessary	flagged − nec.	removable	necessary	removable − nec.	sinks
Claude Haiku 4.5	0.636	0.128	+0.508 [+0.460, +0.553]	0.147	0.169	-0.022 [-0.042, -0.001]	0.130
Gemma-4-31B-IT	0.613	0.082	+0.531 [+0.485, +0.577]	0.103	0.125	-0.022 [-0.040, -0.005]	0.080
Qwen3.6-Flash	0.629	0.061	+0.568 [+0.524, +0.613]	0.082	0.112	-0.030 [-0.043, -0.017]	0.056
Ministral-14B	0.606	0.111	+0.494 [+0.451, +0.537]	0.120	0.151	-0.031 [-0.049, -0.013]	0.122
n tools	591	12,352		3,518	5,694		3,731

Table 26: The size-controlled logit of Table 1 refitted with the number of judge-flagged tools as a covariate and with the judge-necessary-set outcome (every unflagged reference tool selected). Cells are log-odds relative to Star-out with 95% cluster-robust CIs; the last rows give the motif likelihood-ratio test and McFadden R^2. Controls: $|G_q|$, depth, width, log question length, agent (AGB-STRICT: $|G_q|$ and log question length, as in Appendix M).

Motif	AGB: paper model	AGB: + judge-flag count	AGB: judge-necessary outcome	AGB-STRICT: paper model	AGB-STRICT: + judge-flag count
Chain	-1.48 [-2.02, -0.95]	-1.12 [-1.63, -0.60]	-0.94 [-1.47, -0.42]	-2.46 [-3.25, -1.66]	-2.22 [-2.99, -1.45]
Hourglass	-1.20 [-1.73, -0.67]	-0.55 [-1.07, -0.04]	-0.14 [-0.68, +0.41]	-1.74 [-2.46, -1.01]	-1.07 [-1.81, -0.33]
Tree-in	-0.31 [-0.73, +0.10]	+0.11 [-0.31, +0.53]	-0.01 [-0.41, +0.39]	+0.12 [-0.69, +0.93]	+0.29 [-0.51, +1.10]
Tree-out	+0.57 [-0.01, +1.15]	+0.98 [+0.39, +1.56]	+0.89 [+0.35, +1.43]	+0.31 [-0.44, +1.05]	+0.79 [+0.01, +1.57]
judge-flag count	—	-1.25 [-1.50, -1.01]	—	—	-1.32 [-1.91, -0.73]
LRT motif χ^2 (p)	595.7 (2.0e-123)	469.3 (2.7e-96)	327.3 (6.2e-66)	190.7 (5.9e-37)	178.9 (1.8e-34)
McFadden R^2	0.272	0.319	0.202	0.251	0.316

(no answer-removable reference tool, Table 11); the size gradient and the Chain/Hourglass penalties are unchanged there.

Table 27: Size-matched slice: success rate on validated AGB items with $|G_q| \in [5, 8]$, agents pooled (four rows per item). Cells are mean ± half-width of the 95% bootstrap CI over (agent, item) rows; n counts distinct items. Motifs whose sizes lie outside the window have few items here (Tree-in/Tree-out sit at 9–10, Rhombus at 4).

Motif	n items	**Success**
Star-out	145	.843 ±.029
Star-in	161	.817 ±.030
Rhombus	0	—
Hourglass	74	.571 ±.057
Chain	160	.209 ±.031
Tree-out	74	.638 ±.055
Tree-in	26	.647 ±.093
Clique	153	.686 ±.037
Cycle	142	.614 ±.040

Table 28: Surface control: success of the hardest (Tree-in) and easiest (Rhombus) motif within question-length quartiles (words; quartiles over all validated AGB items). Cells are mean ± 95% bootstrap half-width over (agent, item) rows; n counts rows.

Question length	**Tree-in** (n)	**Rhombus** (n)	**Gap**
Q1: 0–65 words	.000 ±.000 (8)	.979 ±.032 (47)	-.979
Q2: 66–84 words	.119 ±.076 (59)	.893 ±.038 (280)	-.774
Q3: 85–110 words	.457 ±.082 (140)	.889 ±.043 (207)	-.432
Q4: >110 words	.470 ±.039 (636)	.875 ±.188 (8)	-.405

Table 29: Per-motif success rate (all reference tools selected, raw recall = 1.0) on AGB (validated items; the two-tool *bridge* items are Chain with $|G_q|$=2). The last column is the macro-average over motifs. Cells are means over records with the half-width of the 95% percentile-bootstrap CI (B=10,000, records resampled within the cell; the macro column resamples within each motif). The *Average* row pools the four agents' items.

Agent	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	Clique	Cycle	Macro avg.
Claude Haiku 4.5	.356 ±.119	.792 ±.167	.866 ±.067	.727 ±.182	.646 ±.125	.559 ±.118	.474 ±.158	.510 ±.143	.565 ±.141	.611 ±.047
Gemma-4-31B-IT	.458 ±.136	.833 ±.146	.897 ±.057	.950 ±.075	.542 ±.146	.651 ±.119	.605 ±.158	.681 ±.138	.717 ±.130	.704 ±.042
Qwen3.6-Flash	.475 ±.136	.833 ±.146	.938 ±.046	1.000 ±.000	.604 ±.146	.588 ±.118	.632 ±.158	.429 ±.143	.587 ±.141	.676 ±.042
Ministral-14B	.424 ±.127	.625 ±.187	.784 ±.082	.818 ±.159	.458 ±.146	.471 ±.118	.474 ±.158	.673 ±.133	.587 ±.141	.590 ±.047
Average	.428 ±.064	.771 ±.083	.871 ±.034	.869 ±.071	.562 ±.068	.566 ±.060	.546 ±.079	.572 ±.067	.614 ±.071	.645 ±.022

Table 30: Success rate (all reference tools selected, raw recall = 1.0) on AGB (validated items) as a function of subgraph size $|G_q|$. Cells are means over records with the half-width of the 95% percentile-bootstrap CI (B=10,000, records resampled within the cell). The *Average* row pools the four agents' items.

Agent	$\|G_q\|$=2	$\|G_q\|$=3	$\|G_q\|$=4	$\|G_q\|$=5	$\|G_q\|$=6	$\|G_q\|$=7	$\|G_q\|$=8
Claude Haiku 4.5	.895 ±.132	.939 ±.076	.686 ±.127	.718 ±.096	.756 ±.122	.605 ±.140	.563 ±.113
Gemma-4-31B-IT	1.000 ±.000	.970 ±.045	.796 ±.112	.654 ±.103	.805 ±.122	.721 ±.128	.704 ±.106
Qwen3.6-Flash	.947 ±.079	.909 ±.106	.837 ±.102	.718 ±.096	.829 ±.110	.651 ±.140	.620 ±.113
Ministral-14B	.842 ±.158	.970 ±.045	.667 ±.127	.590 ±.109	.707 ±.134	.581 ±.140	.535 ±.120
Average	.921 ±.059	.947 ±.042	.745 ±.060	.670 ±.053	.774 ±.064	.640 ±.070	.606 ±.056

N SECOND-CATALOG REPLICATION

Table 31 applies the sampling, generation, soft-validation and selection-only harness to a second, unrelated catalog (ToolLinkOS, 573 tools; audit-only, no executable runtime, Tool Graph taken from the catalog's declared dependencies). The pre-registered test of a transferable nine-way motif ranking fails: Spearman ρ against the AGB ranking of the same agent is .79 for Claude Haiku 4.5 but .41–.63 for the other three agents, largely because the catalog populates only the easy end of the taxonomy (10 rhombi, no directed cycles longer than two, sizes 2–7, four motifs at or near ceiling). What does transfer is the coarse structure: success degrades with $|G_q|$ for every agent, and Tree-in and Chain are among the hardest motifs for every agent. Because the catalog has no output schemas, this replication tests everything except Tool Graph construction.

Table 31: Second-catalog replication (ToolLinkOS, 573 tools, audit-only: no executable runtime, Tool Graph = the catalog's declared dependencies). Complete-set success per motif under the paper's selection-only harness, mean ± 95% bootstrap half-width (B=10,000); n = items per motif; ρ = Spearman correlation between this motif ranking and the AGB ranking of the same agent over motifs with ≥ 10 items here. The catalog has only 10 rhombi and 21 two-cycles. Bottom rows: validator pass rates (V_{schema} is the baseline `name=value` binder; structural redundancy is the tool-level test).

Agent	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	Clique	Cycle	ρ vs. AGB
n	50	50	50	10	50	38	50	50	21	
Claude Haiku 4.5	.860 ±.090	.880 ±.090	.980 ±.030	1.000 ±.000	.940 ±.070	.868 ±.105	.660 ±.130	.980 ±.030	1.000 ±.000	.79
Gemma-4-31B-IT	.860 ±.090	.840 ±.100	.980 ±.030	1.000 ±.000	1.000 ±.000	.947 ±.066	.720 ±.120	.940 ±.070	1.000 ±.000	.41
Qwen3.6-Flash	.820 ±.110	.860 ±.090	.980 ±.030	1.000 ±.000	1.000 ±.000	.921 ±.092	.640 ±.130	.940 ±.070	1.000 ±.000	.63
Ministral-14B	.860 ±.090	.840 ±.100	1.000 ±.000	1.000 ±.000	1.000 ±.000	.947 ±.066	.680 ±.130	.940 ±.070	.952 ±.071	.56
V_{sem}	.68	.70	.80	.70	.66	.71	.68	.74	.52	
V^*_{min}	.86	.62	.84	1.00	.65	.74	.54	.81	.67	
V_{schema} (baseline)	.26	.22	.06	.80	.62	.61	.66	.98	.95	
struct. reducible	.00	1.00	.00	1.00	1.00	.00	.00	1.00	.00	

O KNOWLEDGE-GRAPH SUBSTRATE

To test whether the motif and size effects of Section 5.3 are a property of the dependency structure rather than of tool APIs, we repeat the selection protocol on a second substrate in which nodes are entities of a finance knowledge graph (S&P 500 companies, news items, executives, funds, products, locations and industries; 5,652 entities and 6,686 typed edges exported from a graph database) and edges are its relations. Motifs are sampled from this graph with the same exact-induced check as in Section 3: chain (sizes 3–6; the graph is acyclic and its longest path has six nodes, so clique and cycle do not exist and chains are capped), star-in, star-out (3–10), tree-in (5–10), tree-out, rhombus and hourglass, 100 items per cell (56 for rhombus). Each item is a cover question anchored at one motif node whose answer is the motif's entity and relation sets, so every reference entity is necessary by construction; the candidate pool holds 25 names. Items are rejected when the answer or an intermediate entity is the only candidate of its type, when the answer is not exactly what the question's pattern derives from the context, when a reference name appears in the question, or when a fact is not redistributable; 2,156 items pass. Two conditions differ only in the decoys: in the *explicit-graph* baseline decoys have no edge to the motif; in the *adjacent-decoy* condition up to ten decoys per item are real neighbours of motif nodes reached by an edge the question excludes (wrong direction, or one hop beyond the pattern), with those edges in the context, so the agent must read direction and depth (2,154 paired items). The four agents of Section 4 select from the pool with the candidates' facts and the relations among them visible, then answer on the context of their selection; scoring is set-based, temperature 0, and every row is committed.

The pre-registered test is the one of Appendix N: per agent, Spearman ρ between the motif ranking here and the agent's AGB ranking under the same metric over the seven shared motifs, positive if $\rho \geq .7$ for at least half the agents and chain hardens monotonically with size. It is *neutral* on success under the adjacent condition (ρ .40, .41, .36 and 1.00; Table 32) and *negative* on exact-set match (ρ between $-.07$ and .36). Transfer is partial and the partition is informative. Tree-in is the hardest motif for every agent under every condition and metric, tree-out is next in six of eight cells, and tree-in hardens with size (pooled ρ_{size} $-.81$ to $-.94$ on success; Table 33); the size-controlled regression of Section 5.3 refitted on these rows gives tree-in -2.92 log-odds against star-out at equal size (motif LRT $p \approx 10^{-75}$). Chain, the hardest motif on tool graphs, is the easiest here for every agent ($+1.01$ log-odds against star-out on exact match), and that single inversion holds ρ near .4 for three agents. With the relations written in the context, following a path is trivial; what remains hard is aggregating several incoming branches and rejecting neighbours reached by an edge the question excludes. This is consistent with chain difficulty on tool graphs arising from the inference of dependencies that tool descriptions leave implicit, and it bounds the claim of Section 5.3 to settings where the dependency structure is not given. The adjacent decoys also move the failure mode of the strong agents from omission to inclusion: Claude Haiku 4.5 recovers every reference entity on $\geq .92$ of items in every cell but its exact-set match falls to .21–.60 on the fan-in and fan-out motifs.

P EXECUTION-GROUNDED EVALUATION

Table 34 reports the execution-grounded panel of Section 5.3: 50 validated AGB items per motif, the same 25-candidate lists as the selection harness, and a uniform JSON action protocol in which each agent selects a tool, fills its arguments, calls the StableToolBench runtime, reads the response and finally answers (at most ten calls per item, temperature 0). Provider-side LLM failures (timeouts, output-token limits) count as failures; their rates are recorded with the released trajectories. For Clique and Cycle no call order satisfies every dependency, so the *Order* column is ≈ 0 there by construction. The LLM equivalence judge on final answers agrees on only 5–7% of items and is not used in the text.

The size gradient survives execution; the motif ranking does not. Table 35 bins the same 450 items per agent by $|G_q|$. Exec success falls from the smallest to the largest bin for every agent (.68 → .34, .61 → .42, .63 → .24 and .21 → .02 for Claude Haiku 4.5, Gemma-4-31B, Qwen3.6-Flash and Ministral-14B; record-bootstrap CIs of the differences exclude zero), and Answer success falls with it. The item-level correlation with size is between a half and a quarter of what it is under selection on the same trajectories, because execution adds failure modes (argument filling, protocol errors) that do not scale with size. This is the sense in which the structural ranking transfers to

Table 32: Knowledge-graph substrate: per-motif rate at the motif-comparison size ($|V|$=6; rhombus 4, hourglass 5; 100 items per cell, 56 for rhombus) for the explicit-graph baseline (decoys unconnected to the motif) and the adjacent-decoy condition (decoys linked to the motif by edges the question excludes). Success = every reference entity selected; exact = selected set equals the reference set. Mean ± 95% percentile-bootstrap half-width (B=10,000). ρ = Spearman correlation between the agent's motif ranking here and its AGB ranking under the same metric over the seven shared motifs (AGB has no chain-length cap; clique and cycle do not exist in the KG).

Metric / condition	Agent	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	ρ vs. AGB
success, explicit	Claude Haiku 4.5	1.00 ±0.00	1.00 ±0.00	0.99 ±0.02	0.98 ±0.03	0.99 ±0.02	1.00 ±0.00	0.91 ±0.05	-0.07
	Gemma-4-31B-IT	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	0.92 ±0.05	0.41
	Qwen3.6-Flash	0.96 ±0.03	0.75 ±0.08	0.98 ±0.03	0.91 ±0.08	0.85 ±0.07	0.55 ±0.10	0.22 ±0.08	0.36
	Ministral-14B	0.95 ±0.04	0.99 ±0.02	1.00 ±0.00	0.98 ±0.03	0.96 ±0.03	0.80 ±0.08	0.66 ±0.09	0.86
success, adjacent	Claude Haiku 4.5	0.97 ±0.04	1.00 ±0.00	0.99 ±0.02	0.98 ±0.03	0.99 ±0.02	1.00 ±0.00	0.92 ±0.05	0.40
	Gemma-4-31B-IT	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	0.92 ±0.05	0.41
	Qwen3.6-Flash	0.89 ±0.06	0.78 ±0.08	0.95 ±0.04	0.86 ±0.09	0.80 ±0.08	0.52 ±0.10	0.21 ±0.08	0.36
	Ministral-14B	0.52 ±0.10	0.91 ±0.05	0.97 ±0.03	0.98 ±0.03	0.83 ±0.08	0.74 ±0.08	0.37 ±0.09	1.00
exact, explicit	Claude Haiku 4.5	1.00 ±0.00	1.00 ±0.00	0.99 ±0.02	0.96 ±0.04	0.99 ±0.02	0.99 ±0.02	0.90 ±0.06	0.02
	Gemma-4-31B-IT	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	1.00 ±0.00	0.92 ±0.05	0.41
	Qwen3.6-Flash	0.96 ±0.03	0.75 ±0.08	0.98 ±0.03	0.91 ±0.08	0.85 ±0.07	0.55 ±0.10	0.22 ±0.08	0.36
	Ministral-14B	0.95 ±0.04	0.99 ±0.02	1.00 ±0.00	0.91 ±0.08	0.96 ±0.03	0.79 ±0.08	0.64 ±0.09	0.57
exact, adjacent	Claude Haiku 4.5	0.97 ±0.04	0.60 ±0.10	0.47 ±0.10	0.41 ±0.12	0.88 ±0.06	0.38 ±0.10	0.21 ±0.08	0.11
	Gemma-4-31B-IT	1.00 ±0.00	0.87 ±0.07	0.98 ±0.03	0.95 ±0.06	1.00 ±0.00	0.84 ±0.07	0.79 ±0.08	-0.07
	Qwen3.6-Flash	0.88 ±0.07	0.54 ±0.10	0.73 ±0.09	0.70 ±0.12	0.76 ±0.08	0.45 ±0.10	0.17 ±0.07	0.00
	Ministral-14B	0.52 ±0.10	0.43 ±0.10	0.53 ±0.10	0.36 ±0.12	0.65 ±0.09	0.29 ±0.09	0.09 ±0.05	0.36

Table 33: Knowledge-graph substrate, size axis: rate per $|V|$ within chain (the KG is a DAG whose longest path has six nodes), star-out and tree-in, 100 items per cell. ρ_{size} = Spearman correlation of the rate with $|V|$ (negative = harder with size).

		Chain				Star-out								Tree-in						ρ_{size}	
Metric / condition	Agent	3	4	5	6	3	4	5	6	7	8	9	10	5	6	7	8	9	10	Chain	Star	Tree
success, explicit	Claude Haiku 4.5	0.99	1.00	1.00	1.00	1.00	1.00	1.00	0.99	1.00	1.00	0.99	1.00	0.94	0.91	0.85	0.94	0.86	0.86	0.77	-0.25	-0.44
	Gemma-4-31B-IT	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.89	0.92	0.79	0.90	0.79	0.77	—	—	-0.67
	Qwen3.6-Flash	0.90	0.88	0.97	0.96	0.92	0.99	0.97	0.98	0.98	0.98	0.98	0.98	0.36	0.22	0.24	0.22	0.19	0.22	0.60	0.30	-0.70
	Ministral-14B	0.96	0.91	0.96	0.95	0.99	1.00	1.00	1.00	0.97	0.99	0.98	0.98	0.72	0.66	0.54	0.63	0.50	0.45	-0.21	-0.62	-0.94
success, adjacent	Claude Haiku 4.5	0.96	0.98	1.00	0.97	1.00	1.00	1.00	0.99	1.00	1.00	0.99	1.00	0.90	0.92	0.89	0.91	0.86	0.86	0.40	-0.25	-0.70
	Gemma-4-31B-IT	1.00	0.99	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.93	0.92	0.83	0.89	0.83	0.80	0.26	—	-0.90
	Qwen3.6-Flash	0.88	0.90	0.93	0.89	0.93	0.92	0.95	0.95	0.95	0.95	0.94	0.92	0.35	0.21	0.17	0.11	0.14	0.15	0.40	0.03	-0.77
	Ministral-14B	0.85	0.86	0.90	0.52	0.96	0.97	0.99	0.97	0.94	0.93	0.90	0.81	0.45	0.37	0.25	0.26	0.34	0.30	-0.20	-0.81	-0.49
exact, explicit	Claude Haiku 4.5	0.99	1.00	1.00	1.00	1.00	1.00	1.00	0.99	1.00	1.00	0.99	1.00	0.91	0.90	0.83	0.94	0.86	0.86	0.77	-0.25	-0.35
	Gemma-4-31B-IT	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.89	0.92	0.79	0.90	0.79	0.77	—	—	-0.67
	Qwen3.6-Flash	0.90	0.88	0.97	0.96	0.92	0.99	0.97	0.98	0.98	0.98	0.98	0.98	0.36	0.22	0.24	0.22	0.19	0.22	0.60	0.30	-0.70
	Ministral-14B	0.95	0.91	0.96	0.95	0.99	0.99	1.00	1.00	0.97	0.99	0.98	0.98	0.68	0.64	0.52	0.63	0.50	0.45	0.32	-0.54	-0.94
exact, adjacent	Claude Haiku 4.5	0.79	0.93	0.97	0.97	0.48	0.30	0.46	0.47	0.39	0.52	0.44	0.53	0.22	0.21	0.20	0.24	0.26	0.30	0.80	0.36	0.77
	Gemma-4-31B-IT	0.90	0.93	0.95	1.00	0.97	0.99	1.00	0.98	1.00	0.98	0.98	0.98	0.84	0.79	0.73	0.77	0.68	0.72	1.00	-0.04	-0.89
	Qwen3.6-Flash	0.79	0.82	0.91	0.88	0.57	0.64	0.68	0.73	0.70	0.71	0.68	0.70	0.30	0.17	0.14	0.09	0.13	0.13	0.80	0.57	-0.81
	Ministral-14B	0.69	0.79	0.90	0.52	0.47	0.32	0.45	0.53	0.51	0.60	0.57	0.57	0.10	0.09	0.06	0.06	0.06	0.07	-0.20	0.83	-0.58

execution "in coarse form": the size gradient, not the nine-way ordering. The contrast was fixed after a first look at the per-size means; the report in the repository says so.

Table 34: Execution-grounded evaluation on a per-motif AGB subset (50 validated items per motif, same 25-candidate lists as the selection panel). Agents call tools on the StableToolBench runtime in a uniform JSON action protocol and finish with an answer. *Exec*: finished with no failed call; *Sink*: every reference sink tool called successfully; *Answer*: the sink response has the reference answer's structure; *Order*: reference tools called in a dependency-respecting order. Cells are means ± 95% bootstrap half-width (B=10,000). ρ: Spearman correlation between this agent's motif ranking under Exec and its selection-only success ranking (Table 3). Provider-side LLM failures (timeouts, token limits) count as failures; their rates are in `results/execution_eval.json`. For Clique and Cycle no call order satisfies every dependency, so *Order* is ≈ 0 by construction.

Agent	Metric	Chain	Star-in	Star-out	Rhombus	Hourglass	Tree-out	Tree-in	Clique	Cycle	ρ
Claude Haiku 4.5	Exec	.34 ±.13	.56 ±.14	.50 ±.14	.46 ±.14	.10 ±.09	.40 ±.14	.38 ±.13	.42 ±.14	.62 ±.14	.65
	Sink	.34 ±.13	.70 ±.12	.50 ±.14	.76 ±.12	.34 ±.13	.18 ±.11	.70 ±.12	.52 ±.14	.82 ±.11	
	Answer	.16 ±.10	.42 ±.14	.06 ±.07	.48 ±.14	.12 ±.09	.00 ±.00	.38 ±.14	.46 ±.14	.52 ±.14	
	Order	.31 ±.17	.51 ±.14	.84 ±.12	.56 ±.15	.26 ±.13	.48 ±.14	.00 ±.00	.07 ±.09	.00 ±.00	
Gemma-4-31B-IT	Exec	.48 ±.14	.54 ±.14	.38 ±.14	.48 ±.14	.24 ±.12	.38 ±.13	.48 ±.14	.42 ±.14	.64 ±.14	.11
	Sink	.34 ±.13	.76 ±.12	.40 ±.14	.58 ±.14	.34 ±.13	.24 ±.12	.68 ±.13	.56 ±.14	.76 ±.12	
	Answer	.16 ±.10	.42 ±.14	.08 ±.07	.32 ±.13	.12 ±.09	.00 ±.00	.36 ±.13	.50 ±.14	.56 ±.14	
	Order	.23 ±.15	.32 ±.15	.94 ±.07	.72 ±.14	.31 ±.14	.49 ±.14	.05 ±.06	.05 ±.06	.00 ±.00	
Qwen3.6-Flash	Exec	.42 ±.14	.52 ±.14	.40 ±.14	.48 ±.14	.08 ±.07	.20 ±.11	.32 ±.13	.36 ±.13	.54 ±.14	.50
	Sink	.54 ±.14	.74 ±.12	.54 ±.14	.60 ±.14	.28 ±.12	.16 ±.10	.78 ±.11	.62 ±.13	.76 ±.12	
	Answer	.20 ±.11	.46 ±.14	.14 ±.09	.32 ±.13	.10 ±.08	.00 ±.00	.46 ±.14	.54 ±.14	.50 ±.14	
	Order	.20 ±.12	.50 ±.14	.92 ±.09	.79 ±.13	.35 ±.14	.51 ±.15	.04 ±.05	.06 ±.06	.00 ±.00	
Ministral-14B	Exec	.06 ±.07	.06 ±.07	.12 ±.09	.10 ±.09	.00 ±.00	.00 ±.00	.04 ±.05	.16 ±.10	.20 ±.11	.54
	Sink	.34 ±.13	.56 ±.14	.44 ±.14	.62 ±.14	.12 ±.09	.10 ±.08	.62 ±.14	.68 ±.13	.68 ±.13	
	Answer	.14 ±.09	.26 ±.12	.08 ±.07	.36 ±.13	.00 ±.00	.00 ±.00	.40 ±.14	.62 ±.13	.48 ±.14	
	Order	.15 ±.09	.46 ±.15	.85 ±.10	.70 ±.14	.34 ±.15	.49 ±.14	.05 ±.06	.08 ±.07	.00 ±.00	

Table 35: Size gradient under execution on the per-motif AGB subset of Table 34 (450 items per agent), binned by $|G_q|$ so that every cell has at least 57 items. *Exec*, *Sink* and *Answer* as in Table 34; *Selection* is complete-set recovery read off the trajectory. Cells are means ± 95% bootstrap half-width (B=10,000). ρ: item-level Spearman correlation between $|G_q|$ and the metric; Δ: mean of the first bin minus the last with its record-bootstrap 95% CI. The motif ranking does not survive execution (Table 34); the size gradient does, at roughly half the strength it has under selection.

Agent	Metric	$\|G_q\|$ 2–3	$\|G_q\|$ 4–5	$\|G_q\|$ 6–7	$\|G_q\|$ 8–10	ρ	Δ [95% CI]
Claude Haiku 4.5	Exec	.68 ±.12	.39 ±.08	.46 ±.12	.34 ±.07	-.19	+.34 [+.20, +.48]
	Sink	.74 ±.11	.57 ±.08	.57 ±.12	.42 ±.08	-.22	+.32 [+.17, +.45]
	Answer	.46 ±.12	.30 ±.07	.24 ±.10	.24 ±.07	-.16	+.22 [+.07, +.37]
	Selection	.74 ±.11	.64 ±.07	.50 ±.12	.20 ±.06	-.45	+.54 [+.41, +.67]
Gemma-4-31B-IT	Exec	.61 ±.12	.42 ±.08	.44 ±.12	.42 ±.08	-.11	+.19 [+.04, +.34]
	Sink	.58 ±.12	.50 ±.08	.60 ±.12	.47 ±.08	-.07	+.11 [-.05, +.25]
	Answer	.37 ±.12	.28 ±.07	.22 ±.10	.27 ±.07	-.08	+.10 [-.04, +.24]
	Selection	.60 ±.12	.51 ±.08	.54 ±.12	.30 ±.07	-.24	+.29 [+.14, +.44]
Qwen3.6-Flash	Exec	.63 ±.12	.35 ±.07	.49 ±.12	.24 ±.07	-.24	+.39 [+.24, +.53]
	Sink	.75 ±.11	.52 ±.08	.63 ±.12	.49 ±.08	-.14	+.26 [+.12, +.39]
	Answer	.49 ±.12	.28 ±.07	.26 ±.10	.28 ±.07	-.10	+.21 [+.07, +.36]
	Selection	.74 ±.11	.53 ±.08	.53 ±.12	.19 ±.06	-.40	+.55 [+.41, +.67]
Ministral-14B	Exec	.21 ±.11	.10 ±.04	.07 ±.07	.02 ±.02	-.23	+.19 [+.09, +.30]
	Sink	.79 ±.11	.45 ±.08	.41 ±.12	.38 ±.08	-.25	+.41 [+.28, +.54]
	Answer	.53 ±.12	.23 ±.06	.21 ±.10	.22 ±.07	-.18	+.30 [+.16, +.45]
	Selection	.74 ±.11	.43 ±.08	.35 ±.12	.08 ±.04	-.48	+.66 [+.53, +.78]